GUIA DE MONTAGEM · ARDUINO UNO · 5 SERVOS · LUVA HALL

Mão Robótica com Luva de Sensores

Tudo para montar, ligar e programar a sua mão em duas versões: com fio, que é a garantida, e sem fio com ESP32, que é o upgrade. Também estão aqui as falhas do plano original e o que fazer com cada uma.

CONTROLADOR **Arduino Uno R3 (regra)**	ATUADORES **5 × servo SG90 (do kit)**
SENSORES **5 × Hall SS49E + 5 ímãs**	ENERGIA DOS SERVOS **4 × AA NiMH = 4,8 V**
SEM FIO (OPCIONAL) **2 × ESP32 DevKit V1, ESP-NOW**	REVISÃO DO GUIA **21 out 2026**



O kit que você escolheu: mão e antebraço impressos em 3D e 5 servos SG90. Foto do anúncio.

RESUMO

O que eu faria no seu lugar

Seis decisões. O resto do guia explica cada uma.

ORDEM

Faça primeiro a versão com fio

Luva ligada por cabo direto no Uno. Ela tem menos peças e menos chances de falhar, e é ela que salva a sua apresentação se o sem fio der problema.

ENERGIA

Servos nas pilhas, Uno na USB

Use 4 pilhas AA recarregáveis (4,8 V) só para os servos. O Uno só manda o sinal. O GND das pilhas tem que ser ligado no GND do Uno.

SENSOR

SS49E analógico

Custa praticamente o mesmo que o A3144, funciona em 3,3 V e 5 V e permite escolher o ponto de fechar no código. O A3144 só diz sim ou não.

POSIÇÃO

Um de cada lado do nó do dedo

Sensor nas costas da mão, logo antes do nó. Ímã na falange proximal, logo depois. O sinal acompanha o movimento inteiro e nenhum fio passa por articulação.

SEM FIO

2 ESP32 com ESP-NOW

Um ESP32 na luva e outro junto da mão. O segundo repassa os dados para o Uno por um fio. O Uno continua sendo o cérebro dos servos, então a regra é cumprida.

DIA DA APRESENTAÇÃO

Leve o cabo da versão 1

Se o rádio falhar no meio da feira, troque o firmware e ligue a luva por fio em 5 minutos.

CRÍTICA

Onde o seu plano está fraco

Você pediu para eu ser duro. Estes são os pontos que podem derrubar o projeto, do mais grave ao menos grave.

● GRAVE

O prazo é apertado: uns 20 dias até 10 de novembro

A entrega das peças leva de 3 a 7 dias, então sobram umas duas semanas de montagem. Para quem está começando, eu estimo de 4 a 6 semanas para o projeto completo. A versão com fio cabe nesse prazo. A sem fio só cabe se nada der errado, e em projeto de robótica alguma coisa sempre dá errado.

O dia 15 de novembro de 2026 cai num domingo e é feriado. Confirme a data com o professor e planeje para o dia 10.

O que fazer: compre tudo nesta semana, de vendedores com entrega rápida, e siga o calendário. A data de corte é **4 de novembro**: se a versão 1 não estiver funcionando até lá, esqueça o ESP32.

● GRAVE

“Só para ficar bonito” não é motivo para colocar o ESP32

A banca vai perguntar por que o projeto tem três placas se uma resolve. “Fica mais tecnológico” é uma resposta fraca. O ESP32 também adiciona duas placas, um rádio, uma segunda fonte de energia e um protocolo de comunicação. Cada um desses pode falhar no dia.

O que fazer: tenha um motivo de engenharia. O sem fio deixa o operador longe da mão, como em robôs que trabalham em lugares perigosos, desarmam bombas ou

fazem cirurgia à distância. E só comece o sem fio depois que a versão com fio estiver funcionando.

● MÉDIO

Ímã que só se aproxima do sensor vira um botão liga/desliga

O campo de um ímã pequeno cai muito rápido com a distância. Ele só vira sinal útil a menos de 1 a 1,5 cm do sensor. Se o ímã ficasse na ponta do dedo e o sensor na palma, como no plano inicial, o sensor não perceberia nada em quase todos os 9 cm do movimento, e o dedo robótico fecharia de uma vez.

O que fazer: use a posição da seção Luva, com o sensor e o ímã a poucos milímetros um do outro, um de cada lado do nó do dedo. Ao dobrar, o ímã gira e se afasta aos poucos, e o sinal muda durante o movimento inteiro. Ela tem limitações próprias, que estão explicadas lá.

● GRAVE

Não havia plano de energia

Cinco SG90 travados podem puxar mais de 3 A. O pino 5V do Uno na USB entrega uns 500 mA. Ligando os servos nele, o Uno reinicia sem parar e parece defeito no código. Muita gente perde dias procurando um “bug” que na verdade é falta de energia.

O que fazer: veja a seção Energia. Use pilhas separadas, GND comum e um capacitor de 1000 µF.

● MÉDIO

O Uno não tinha como receber o sinal do ESP32

O Uno não tem Wi-Fi nem Bluetooth. Do jeito que estava, a luva transmitia e ninguém recebia.

O que fazer: use um segundo ESP32 ao lado da mão e ligue a saída TX dele no pino D0 do Uno.

● MÉDIO

O ímã do dedo vizinho pode atrapalhar

Os nós dos dedos ficam a uns 2 cm um do outro. Com o dedo dobrado, o seu ímã se afasta do sensor para mais ou menos 1,5 cm, enquanto o ímã do vizinho continua a uns 2 cm. No fim do movimento, o vizinho pesa quase tanto quanto o seu. Isso faz um dedo “puxar” um pouco o outro.

O que fazer: use ímã pequeno (5×2 mm), deixe cada sensor bem perto do próprio ímã e teste um dedo de cada vez no Serial Plotter antes de colar. Num punho normal todos os dedos dobram juntos, o que disfarça o efeito. Na demonstração de um dedo sozinho ele aparece mais.

● MÉDIO

O SG90 é fraco e tem engrenagem de plástico

O SG90 serve para demonstração, mas não para segurar objetos pesados. Se o código mandar o servo além do que o dedo consegue ir, ele fica forçando, esquenta, gasta pilha e pode espanar a engrenagem.

O que fazer: calibre os ângulos de cada dedo com o sketch `teste_servos` e tenha 1 ou 2 servos de reserva. O MG90S tem engrenagem de metal e quase o mesmo tamanho, mas pode exigir um ajuste no suporte. Em 4,8 V a força é igual à do SG90. O que você ganha é durabilidade.

● ATENÇÃO

A mecânica vai dar mais trabalho que a eletrônica

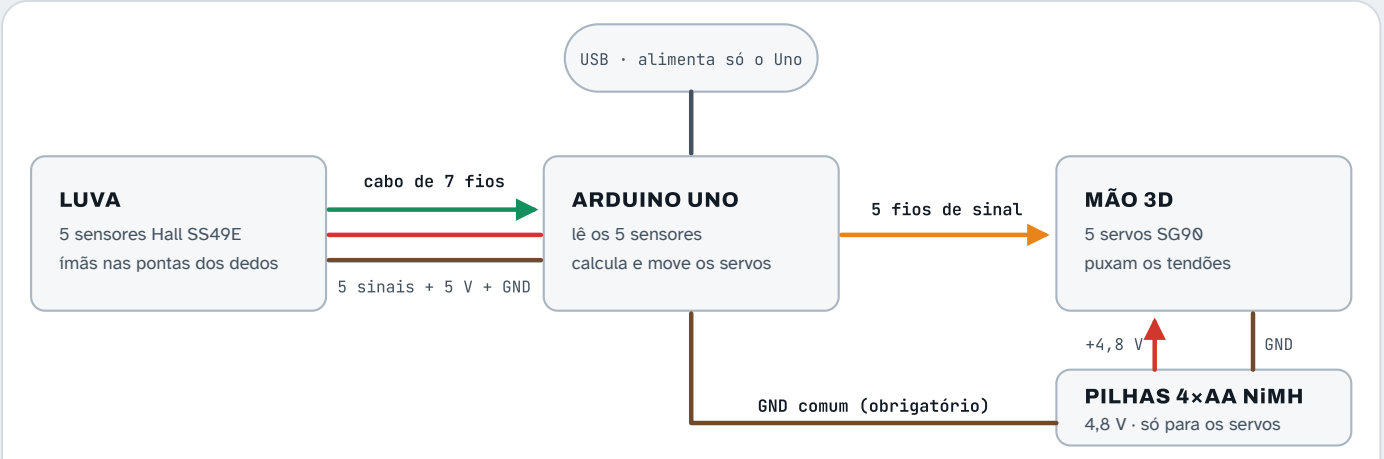
Em mãos robóticas com tendão, os problemas reais são linha que estica, nó que escorrega, dedo que não volta e servo montado no ângulo errado. Reserve pelo menos uma semana só para isso.

VISÃO GERAL

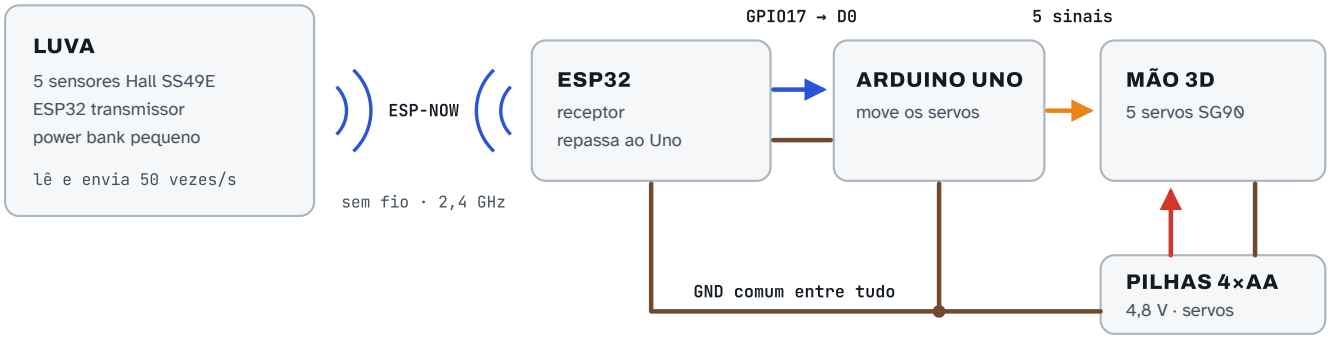
Como o sistema funciona

Nas duas versões o caminho é o mesmo: o sensor percebe o ímã, uma placa calcula o quanto cada dedo fechou e o Uno move o servo correspondente. A diferença é como a informação sai da luva.

+ positivo (5 V / 4,8 V) +3,3 V GND (negativo) sinal dos servos
saída dos sensores dados seriais



Versão 1, com fio. O Uno lê os sensores pelas entradas analógicas e move os servos. As pilhas só alimentam os servos. O fio marrom embaixo liga o GND das pilhas ao GND do Uno. Sem ele, o sinal não tem referência e os servos tremem.



Versão 2, sem fio. A luva calcula o quanto cada dedo fechou (0 a 100 %) e manda pelo ar. O ESP32 receptor só repassa esses números ao Uno por um fio serial. O Uno faz exatamente o mesmo trabalho da versão 1: converte para ângulo e move os servos.

	VERSÃO 1 · COM FIO	VERSÃO 2 · SEM FIO
Placas	1 (Uno)	3 (Uno + 2 ESP32)
Custo extra	só sensores, ímãs e pilhas	+ 2 ESP32 e um power bank (veja Compras)
Dificuldade	baixa / média	média / alta: duas IDEs de placa, rádio, serial
O que pode falhar	fios, sensores, energia	tudo da versão 1 + rádio, bateria da luva, ligação serial
Impressiona a banca?	sim, se funcionar liso	mais, se funcionar; menos que a versão 1 se travar

COMPRAS

O kit e a lista de compras

Os preços são faixas aproximadas pesquisadas em outubro de 2026. Confira antes de comprar, porque eles mudam toda semana.

O que dá para saber do anúncio

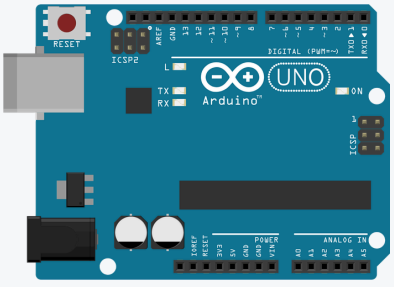
- **Preço:** R$ 227 (6× R$ 37,83), com frete grátis aparecendo em algumas buscas. Nota 4,8 com só cerca de 10 avaliações.
- Existe um anúncio irmão, “Kit Completo Mão Robótica 3d Com Arduino”, por R$ 327. A diferença deve ser a placa. Se você ainda não tem um Uno, compare. Comprado à parte, um Uno compatível sai por R$ 35–79.
- Os SG90 azuis vendidos assim quase sempre são clones, mais frágeis que o TowerPro original.
- Uma das buscas mostrou a imagem do anúncio com código de anúncio argentino (MLA) em vez de brasileiro (MLB). Não consegui confirmar, mas confira se o envio é nacional para não esperar semanas nem pagar imposto de importação.

Não consegui abrir a página do anúncio daqui, então os itens acima vêm dos trechos que aparecem nas buscas.

Pergunte ao vendedor antes de comprar

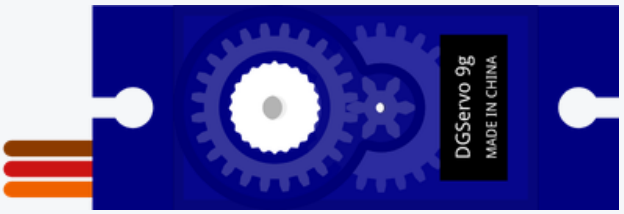
1. A mão vem montada ou em peças?
2. Vem linha para os tendões? E elásticos ou molas para os dedos abrirem?
3. Vem parafusos e os braços (horns) dos servos?
4. Os servos são SG90 de 180° (de posição), e não de rotação contínua?
5. Tem manual, vídeo ou arquivo de montagem?
6. O envio é do Brasil e com nota fiscal?

Leia também as perguntas que outros compradores já fizeram no anúncio. Um projeto parecido do IFBA (mão 3D, Uno e 5 SG90) usou linha de nylon para fechar e elástico para abrir. Se o kit não trouxer, a lista abaixo inclui os dois.



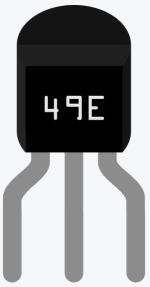
Arduino Uno R3

O cérebro dos servos. Obrigatório pela regra.



Micro servo SG90

Vem no kit. Fios: marrom GND, vermelho +, laranja sinal.



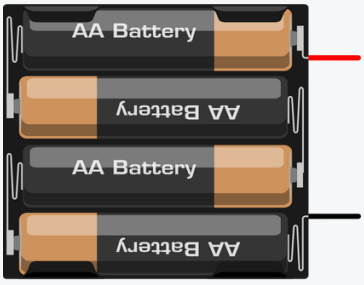
Sensor Hall SS49E

Um por dedo. Pernas VCC, GND e OUT.



Ímã de neodímio

5×2 ou 6×3 mm, colado na ponta de cada dedo da luva.



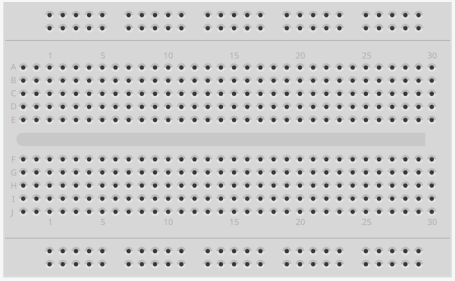
Suporte 4 × AA

Com pilhas NiMH, alimenta só os servos.



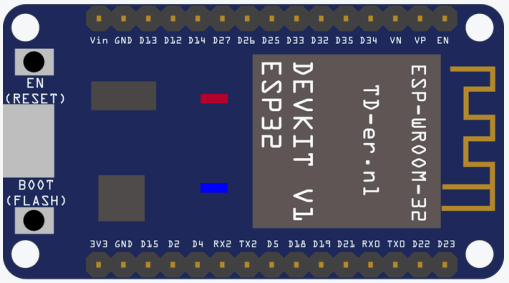
Capacitor 1000 µF

Segura a tensão quando os servos arrancam. A faixa clara é o (–).



Protoboard

Os trilhos + e – viram as linhas de energia dos servos.



ESP32 DevKit V1

Só na versão 2: um na luva, outro junto da mão.

Ilustrações no estilo Fritzing. Os créditos estão no fim da página.

Lista de compras

ITEM	QTD.	PREÇO APROX.	PARA QUÊ / CUIDADO	ONDE
AS DUAS VERSÕES				

ITEM	QTD.	PREÇO APROX.	PARA QUÊ / CUIDADO	ONDE
Sensor Hall linear SS49E / 49E TO-92, 3 pernas	kit de 10	R$ 18–22	Um por dedo, com reservas. Confira que o anúncio diz **linear** (49E). 41F, 44E e 3144 são digitais.	ML, Eletrogate, EasyTronics
Ímãs de neodímio disco 5×2 mm ou 6×3 mm	lote de 50	R$ 18–31	Um por dedo. Se o sinal ficar fraco, empilhe dois. Mantenha longe do celular e de cartões.	ML, Lorben
Luva fina algodão, helanca ou ciclismo	1 par	R$ 5–40	A de algodão é mais fácil de costurar. A de ciclismo é mais firme e fica mais bonita.	Ferragens/EPI, Decathlon, ML
Suporte 4 × AA com chave	1	R$ 10–17	Já vem com chave liga/desliga.	Eletroparts, Fermarc, ML
Pilhas AA NiMH 2500 mAh	4	R$ 32–60	Use NiMH. Alcalinas novas passam do limite do SG90.	Amazon, Nagem, ML
Carregador NiMH	1	R$ 56–106	Só se você não tiver um. O combo carregador + 4 pilhas sai por R$ 106–118.	Elgin, Kalunga, Amazon
Capacitor eletrolítico 1000 µF 16 V	2	R$ 1–5	Um vai na linha dos servos e o outro é reserva. Respeite a polaridade.	Baú da Eletrônica, Eletrogate
Protoboard 830 pontos	1	R$ 11–16	Distribui o + e o GND dos servos.	Eletrogate, MakerHero
Jumpers macho-macho e macho-fêmea	1 pacote de cada	R$ 16–24	O conector do servo encaixa direto no jumper macho.	Eletrogate, MakerHero
Barra de pinos 1×40	2–4	R$ 2–5	Ponta do cabo da luva e conexão dos servos.	Baú da Eletrônica, Shopee
Cabo flat 26 AWG	2 m (v1) · 1 m (v2)	R$ 10–26	Cabo fino e flexível da luva. Um cabo de rede velho (8 vias) também serve.	Cinestec, Eletrodex, ML
Botão táctil 6×6 mm (push button)	2–3	≈ R$ 1–5 o pacote	No D2 do Uno: calibra e liga o modo demonstração sem computador. Preço estimado, não pesquisado.	Baú da Eletrônica, Eletrogate
Termo-retrátil sortido	1 kit	R$ 15–30	Isola as pernas dos sensores e as emendas.	Shopee, Magalu, ML
SÓ A VERSÃO 2				

ITEM	QTD.	PREÇO APROX.	PARA QUÊ / CUIDADO	ONDE
ESP32 DevKit V1 ESP-WROOM-32, 30 pinos	2	R$ 66–125	Tem que ser o ESP32 **clássico** (WROOM-32). C3, S2 e S3 têm outros pinos. Compre os dois iguais.	ML, Usinainfo, Eletrogate
Cabo USB de dados	1–2	R$ 15–42	Muitos cabos de carregador são “só carga” e o PC não reconhece a placa.	KaBuM, Pichau ou o do celular
Power bank pequeno	1	R$ 0–170	O do seu celular serve. Teste se ele não desliga sozinho.	ML, Amazon
Jumpers fêmea-fêmea, velcro ou pulseira	1	R$ 10–30	Para prender o ESP32 e o power bank no pulso.	Armarinho, ML
OPCIONAIS E ALTERNATIVAS				
Fonte 5 V 3 A (P4) + adaptador P4 com borne	1	R$ 26–47	Substitui pilhas, suporte e carregador se a apresentação for perto de tomada. Junte o GND da fonte com o GND do Uno.	AutoCore, Usinainfo
Linha para tendão	1 carretel	R$ 12–100	Só se o kit não trouxer. A multifilamento quase não estica. A monofilamento é barata e serve.	Loja de pesca, Magalu
Elásticos finos	1 pacote	R$ 5–15	Só se o kit não tiver retorno dos dedos.	Farmácia, armarinho
Servo reserva SG90 ou MG90S 180°	1–2	R$ 15–39	Muito MG90S anunciado é **360°** (rotação contínua) e não serve. Procure “180°” ou “de posição”.	ML, Usinainfo
Sensor A3144 (no lugar do SS49E)	5–7	R$ 1–4 cada	Só liga/desliga. Precisa de VCC ≥ 4,5 V.	EasyTronics, AutoCore
Resistores 10 kΩ	5–10	R$ 3–10	Só para o A3144 no ESP32. Com o SS49E não precisa de nenhum.	Baú da Eletrônica
Arduino Uno R3 + cabo USB	1	R$ 35–79	Se precisar comprar. A versão DIP (chip removível) é a mais parecida com o original.	Eletrogate, Usinainfo

VERSÃO 1 · SEM O KIT E SEM O UNO

≈ R$ 195–380

Com pilhas, suporte e carregador. Se você já tem carregador: R$ 140–275. Com fonte 5 V no lugar das pilhas: R$ 120–245.

VERSÃO 2 · A MAIS

≈ R$ 135–370

Dois ESP32, cabos e power bank. Reaproveitando o power bank e o cabo do celular: R$ 80–160.

FRETE

+ R$ 15–35 por loja

Compre os componentes numa loja só e pegue sensores e ímãs no ML com frete grátis. O AliExpress é mais barato, mas demora semanas e paga ICMS.

ENERGIA

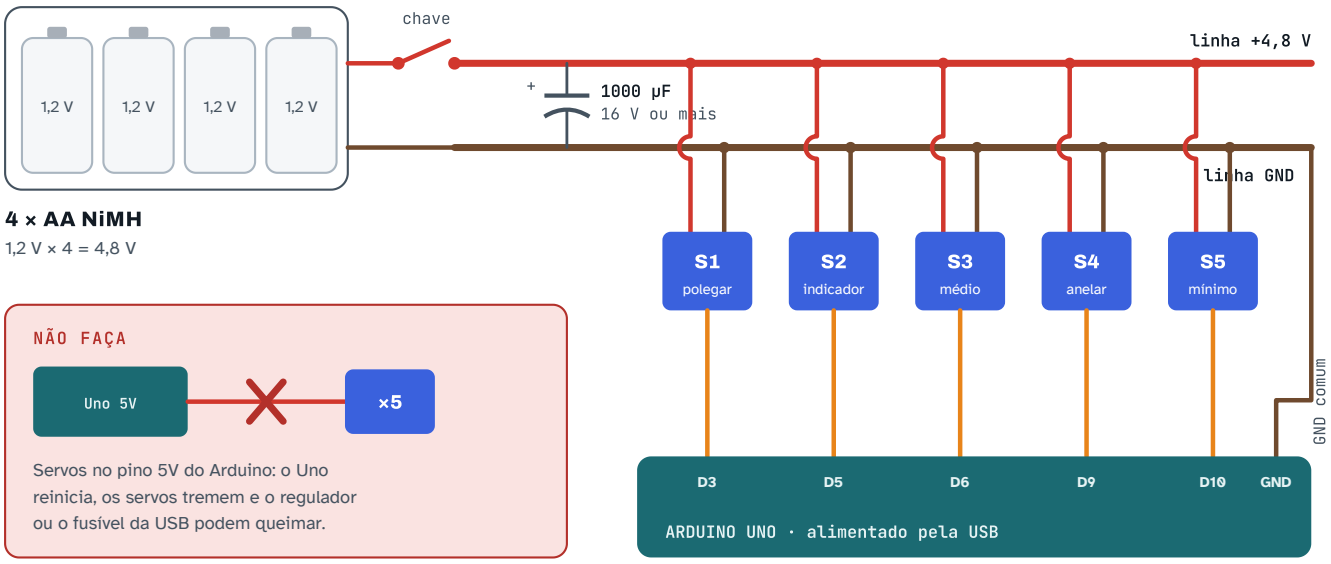
Como alimentar os 5 servos

Esta é a parte que você disse que não sabia fazer, e é a que mais queima projetos. O Arduino controla os 5 servos tranquilamente. O que ele não consegue é fornecer a corrente que eles puxam.

O QUE PUXA CORRENTE	POR SERVO	5 SERVOS	COMENTÁRIO
SG90 parado, segurando a posição	5–10 mA	≈ 50 mA	Quase nada.
SG90 se movendo sem carga	100–250 mA	0,5–1,25 A	Já passa do que a USB do Uno entrega.
SG90 travado (forçando o dedo)	360–750 mA	até ≈ 3,5 A	O pior caso. Use 0,7 A por servo para dimensionar.

O que o Uno entrega no pino 5V: uns 500 mA pela USB, porque há um fusível de 500 mA na placa e a porta do PC também limita. Pelo jack de 9 V, só uns 200–300 mA antes de o regulador esquentar.

Os valores de corrente do SG90 vêm de medições publicadas e de lojas. A TowerPro não publica número oficial, e os clones variam.



Como ligar. As pilhas formam duas linhas, + e GND, que podem ser duas fileiras da protoboard ou uma barra de pinos. Cada servo pega + e GND dessas linhas, e só o fio laranja vai ao Arduino. O GND do Arduino também entra na linha GND. Onde um fio vermelho “pula” por cima do marrom, os dois não se tocam.

Passo a passo

1. Coloque 4 pilhas NiMH carregadas no suporte com chave.
2. Ligue o vermelho do suporte numa fileira da protoboard (linha +) e o preto na outra (linha GND).
3. Coloque o capacitor entre as duas fileiras. A perna do lado da faixa clara (–) vai na linha GND.
4. Ligue o vermelho de cada servo na linha + e o marrom na linha GND.
5. Ligue o laranja de cada servo no pino do Uno: D3, D5, D6, D9, D10.
6. Ligue um jumper de qualquer pino GND do Uno até a linha GND.
7. Ligue o Uno na USB e só então ligue a chave das pilhas.

Os erros que queimam peça

- **Esquecer o GND comum.** Os servos ficam doidos ou não se mexem.
- **Pilha alcalina nova** passa de 6 V com 4 em série, acima do limite do SG90. Use NiMH (4,8 V).
- **Bateria de 9 V** (a quadradinha) é fraca demais para servos e acaba em minutos. Não use nem para o Uno.
- **Capacitor ao contrário** esquenta e pode estourar. Confira a faixa (–).
- **Inverter o + e o GND do pack** queima servos na hora. Confira com multímetro antes de ligar os servos.

Outras fontes de energia, comparadas

OPÇÃO	TENSÃO	VEREDITO	POR QUÊ
4 × AA NiMH	4,8 V	● USE ESTA	Tensão dentro da faixa do SG90, aguenta os picos e é recarregável. Não precisa de regulador.
Fonte 5 V 3 A na tomada	5,0 V	● BOA NA BANCADA	Barata e nunca acaba. Só serve se houver tomada perto da mesa da apresentação.
4 × AA alcalina	até ~6,4 V	● EVITE	Novas passam do máximo do SG90. Depois caem rápido com a corrente dos servos.
Power bank USB	5,0 V	● SÓ PARA TESTE	Muitos cortam a saída no pico de corrente ou quando o consumo cai, e a mão “desliga” sozinha.
LiPo 2S + regulador UBEC 5 V	7,4 → 5 V	● AVANÇADO	Mais forte e leve, mas precisa de carregador balanceado e pode pegar fogo se for mal usada. Não vale o risco aqui.
Bateria 9 V	9 V	● NÃO USE	Tensão alta demais para servo e capacidade minúscula. Acaba em minutos.

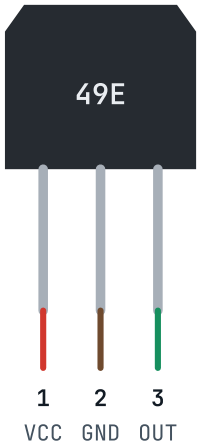
SENSORES

Qual sensor Hall usar, e por quê

Um sensor Hall mede o campo magnético. Perto de um ímã, o sinal muda. Existem dois tipos comuns, e a escolha muda o código e a ligação.

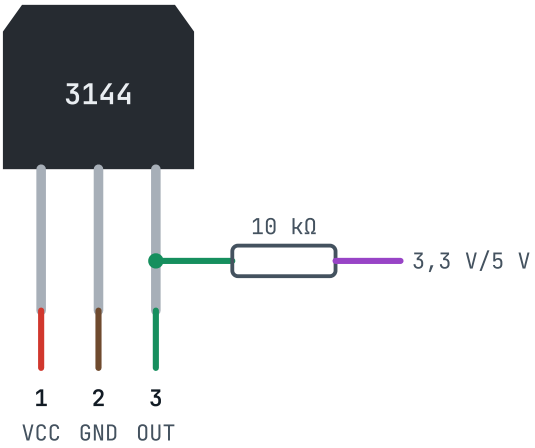
	SS49E (49E) · ANALÓGICO LINEAR	A3144 (3144) · DIGITAL
O que entrega	Uma tensão que varia com o campo. Sem ímã fica em ~metade do VCC e sobe ou desce conforme o polo.	Só dois estados: ligado (saída em 0 V) ou desligado.
Alimentação	2,7 a 6,5 V no original (clones: 3,0 V ou até 4,5 V mínimo). Funciona no Uno (5 V) e no ESP32 (3,3 V).	4,5 a 24 V. Não funciona direito em 3,3 V. No ESP32, alimente pelo VIN (5 V).
Saída	Liga direto no pino analógico.	Coletor aberto: precisa de resistor pull-up.
Polo do ímã	Qualquer um. O código calibra.	Só o polo sul, virado para a face com a escrita.
No código	Acompanha o movimento inteiro, se o ímã e o sensor ficarem um de cada lado do nó do dedo.	Aberto ou fechado. Aciona entre 70 e 350 G, conforme a peça, e você não escolhe.
Preço	R$ 18–22 o kit de 10	R$ 1–4 cada
Veredito	● RECOMENDADO	● SÓ SE JÁ TIVER

vista de frente: face com a escrita virada para você



SS49E · analógico

VCC em 3,3 V ou 5 V. OUT vai direto no pino analógico.



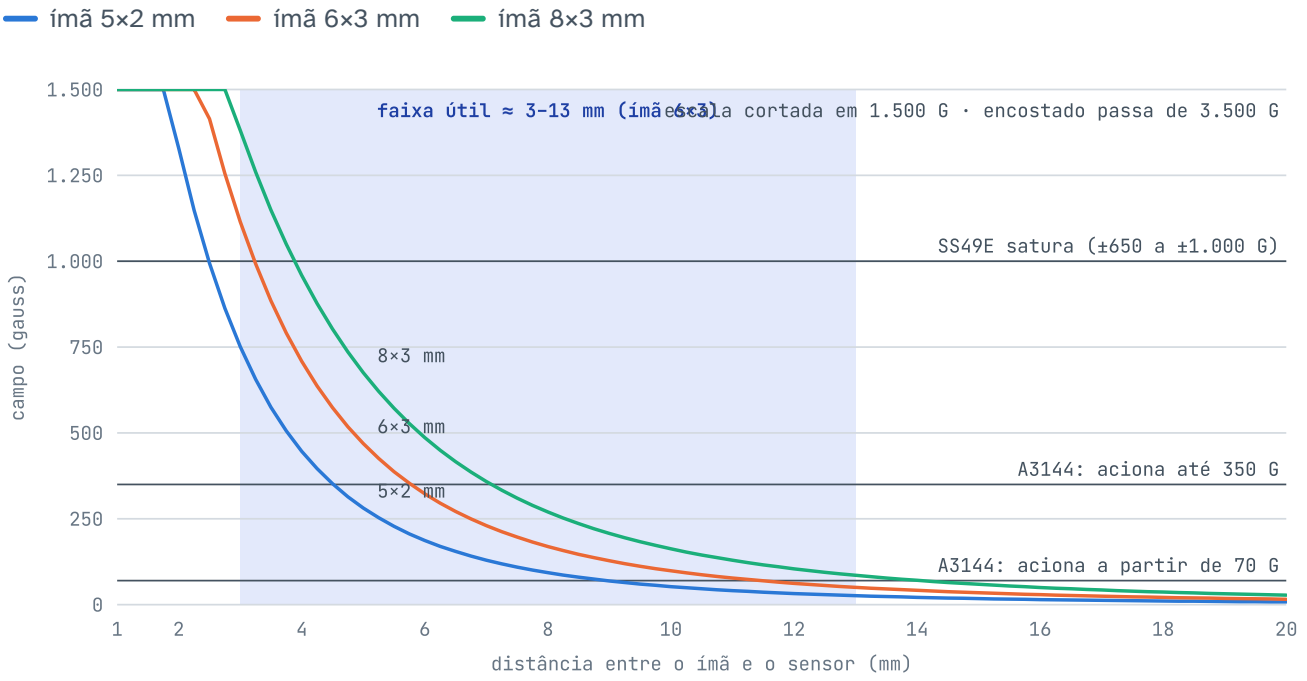
A3144 · digital

VCC em 5 V. OUT precisa de pull-up.

Pinagem. Os dois têm a mesma ordem de pinos olhando pela face com a escrita. O A3144 tem saída “coletor aberto”: ele só puxa o fio para o GND, então precisa de um resistor (pull-up) que puxe para cima. No Uno basta o `INPUT_PULLUP` interno. No ESP32 os pinos 34 a 39 não têm pull-up interno, então o resistor externo é obrigatório.

Campo do ímã × distância até o sensor

Campo no eixo de ímãs de neodímio N35 (cálculo pela fórmula do cilindro magnetizado). Passe o dedo ou o mouse para ver os valores.



▼ Ver os números em tabela

DISTÂNCIA	ÍMÃ 5×2 MM	ÍMÃ 6×3 MM	ÍMÃ 8×3 MM
1 mm	2.361 G	2.878 G	2.764 G
2 mm	1.329 G	1.802 G	1.985 G
3 mm	751 G	1.115 G	1.381 G
4 mm	447 G	709 G	959 G
5 mm	282 G	469 G	676 G
6 mm	187 G	323 G	486 G
8 mm	93 G	169 G	270 G
10 mm	53 G	99 G	162 G
12 mm	32 G	62 G	104 G
15 mm	18 G	35 G	59 G
20 mm	8 G	16 G	28 G

O que o gráfico mostra

Encostado, o ímã passa de 3.500 gauss. A 1 cm, sobram uns 100 G. A 2 cm, quase nada. O SS49E satura perto de ±1.000 G (no mínimo ±650 G), então na prática a faixa útil vai de uns **3 mm a 13 mm** com o ímã 6×3 mm.

Repare que o elemento sensor fica cerca de 0,5 mm dentro do corpo do sensor. Mesmo "encostado", a distância nunca é zero.

Na prática, para a sua luva

- Na posição do nó do dedo, deixe o ímã a **4–8 mm** do sensor com o dedo esticado. Ao dobrar, ele vai para ~15 mm, e o sinal percorre quase toda a faixa útil.
- No Uno, cada passo do ADC vale uns 3,5 G. No ESP32 vale ~1 G, mas o ruído é de 10 a 20 passos. Por isso o código faz média das leituras.
- Use o **Serial Plotter** da Arduino IDE para ver a curva de cada dedo antes de colar os sensores de vez.
- Se o sinal ficar fraco, troque por um ímã maior ou empilhe dois de 5×2.

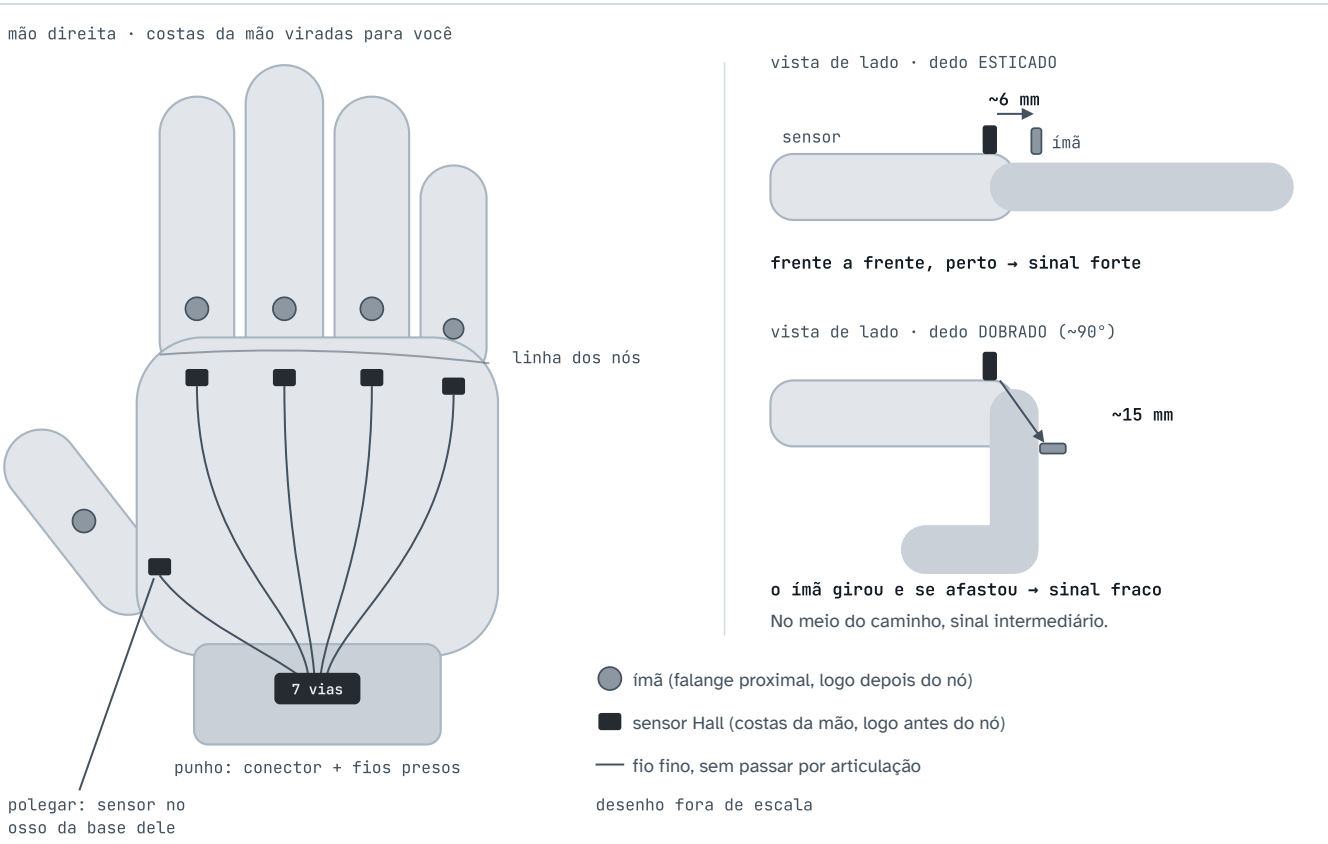
CLONES DO 49E

O SS49E original da Honeywell funciona a partir de 2,7 V. Alguns clones vendidos como "49E" pedem no mínimo 4,5 V. Se você for fazer a versão sem fio, **teste um sensor ligado em 3,3 V antes de montar a luva**. Sem ímã, a saída tem que ficar perto de 1,65 V e mudar quando o ímã se aproxima.

LUVA

Onde colocar sensores e ímãs

Você quer que a mão robótica acompanhe o dedo do começo ao fim, e não só no final. Para isso, o ímã e o sensor têm que ficar a poucos milímetros um do outro o tempo todo, um de cada lado de **uma** articulação. A melhor articulação é a da base do dedo (MCF, o "nó"). Ela dobra uns 85–90° quando você fecha a mão.



Um par por dedo, atravessando o nó. O sensor fica nas costas da mão, que não dobra, então o fio dele nunca passa por uma articulação. O ímã fica na falange proximal e não tem fio. Com o dedo esticado, os dois se olham de frente a uns 6 mm. Ao dobrar, o ímã gira junto com o dedo e se afasta, e o sinal cai de forma contínua até o punho fechado. Uma luva publicada em 2026 (arXiv) usa o mesmo princípio de um par por articulação.

DEDO	SENSOR (FIXO, COM FIO)	ÍMÃ (MÓVEL, SEM FIO)	DOBRA DA ARTICULAÇÃO
Indicador, médio, anelar, mínimo	Costas da mão, em cima do osso do metacarpo, logo antes do nó. Face da escrita virada para o dedo.	Costas da falange proximal, logo depois do nó. Face plana virada para o sensor.	~85–90°
Polegar	Costas do osso da base do polegar (1º metacarpo), antes da articulação do meio do polegar.	Costas da falange proximal do polegar.	~50–60°

POR QUE NÃO O SENSOR NO OSSO DISTAL E O ÍMÃ NO PROXIMAL?

Porque volta o problema do botão liga/desliga. Com o dedo esticado, a ponta fica a uns 5–6 cm da falange proximal, longe demais para o sensor. O sinal só aparece no fim da dobra. E o osso distal é o que mais se mexe, então o fio do sensor passaria por cima de duas articulações e quebraria na solda. O mesmo vale para o sensor na ponta e o ímã na palma: a regra é ímã na parte que se mexe, sensor na parte fixa, os dois perto um do outro.

LIMITAÇÕES DESTA POSIÇÃO

- **Ela mede a dobra do nó.** Num punho normal o nó dobra ~90° e funciona. Se você dobrar só as pontas dos dedos (“garra”) com o nó reto, o sensor quase não vê.
- **A curva não é reta:** o sinal muda mais rápido no começo da dobra que no fim. A calibração de aberto e fechado já resolve boa parte. Se o meio do movimento ficar estranho, dá para acrescentar um terceiro ponto de calibração (meio dobrado).
- **A luva precisa ser justa** (lycra ou ciclismo). Se ela escorregar sobre o nó, a distância muda e a leitura varia sem o dedo mexer.
- **O ímã do vizinho atrapalha um pouco** no fim da dobra. Use ímã 5×2 mm e teste dedo por dedo.

Teste antes de colar qualquer coisa (30 min)

1. Prenda com fita um sensor e um ímã no **indicador**, nas posições da tabela.
2. Grave o `teste_sensores` e abra *Ferramentas* → *Serial Plotter* em 115200.
3. Dobre o dedo devagar, de esticado até o punho fechado.
4. A curva tem que ser **lisa do começo ao fim**, sem ficar reta (achatada) em nenhuma ponta.
5. Ajuste a distância entre 4 e 8 mm e teste o ímã virado dos dois lados. Fique com a combinação de maior variação.
6. Repita para cada dedo e marque as posições com caneta.

Achatada no começo quer dizer que o sensor saturou: afaste o ímã. Achatada no fim quer dizer que o ímã foi longe demais: aproxime. Para este teste use o programa `teste_sensores`, logo abaixo: ele não precisa de servos nem de pilhas.

Como montar a luva

1. Use luva fina e **justa**. Luva frouxa escorrega e muda a leitura.
2. Dobre as pernas do sensor para ele ficar “em pé”, com a face da escrita virada para o dedo. Isole cada perna com termo-retrátil.
3. Cole o sensor nas costas da mão com cola quente sobre um pedacinho de feltro, ou costure.
4. Cole o ímã em pé na falange proximal, de frente para o sensor. Todos os ímãs com o **mesmo polo** virado para o sensor.
5. Junte os 5 VCC num fio e os 5 GND noutro. Saem 7 fios: VCC, GND e 5 sinais.
6. Leve os fios pelas costas da mão até o punho e prenda com costura, para um puxão não arrancar as soldas.

`teste_sensores/teste_sensores.ino`

Arduino Uno · teste da luva · 95 linhas

Mostra os 5 sensores no Serial Plotter, para você achar a melhor posição do ímã antes de colar. Digite t para ver o resumo (mínimo, máximo, variação).

```
/*
  ============================================================================
  TESTE DOS SENSORES  -  para achar a melhor posicao do ima (Arduino Uno)
  ============================================================================
  Para que serve:
    Antes de colar qualquer coisa na luva, prenda o sensor e o ima com fita
    e use este programa para ver se o sinal acompanha o dedo do comeco ao fim.
    Nao precisa de servo nem de pilha: so o Uno na USB e os sensores.

  Ligacoes (iguais as da versao 1):
    SS49E: VCC -> 5V | GND -> GND | OUT -> A0 (polegar) A1 A2 A3 A4 (minimo)
    Pode testar com 1 sensor so (no A0): os outros mostram valores soltos.

  Como usar:
    1. Grave e abra Ferramentas -> Serial Plotter, em 115200.
    2. Dobre o dedo DEVAGAR, de esticado ate o punho fechado, varias vezes.
    3. O que voce quer ver: uma curva LISA, que muda do comeco ao fim.
         - Fica reta (achatada) no comeco? O ima esta perto demais: afaste.
         - Fica reta no fim?               O ima esta longe demais: aproxime.
         - Variacao menor que ~80?         Ima mais forte ou mais perto.
    4. Abra o Monitor Serial e digite t para ver o resumo em texto
       (minimo, maximo e variacao de cada sensor). Digite r para zerar.
  ============================================================================
*/

const uint8_t PINO_SENSOR[5] = { A0, A1, A2, A3, A4 };
const char*   NOME_DEDO[5]   = { "polegar", "indicador", "medio", "anelar", "minimo" };
const int     VARIACAO_BOA   = 80;    // minimo para o movimento ficar suave (quanto mais, melhor)

int minimo[5];
int maximo[5];
bool modoTexto = false;
unsigned long proximoPrint = 0;

int lerSensor(uint8_t i) {
  int soma = 0;
  for (uint8_t k = 0; k < 4; k++) soma += analogRead(PINO_SENSOR[i]);
  return soma / 4;
}

void zerar() {
  for (uint8_t i = 0; i < 5; i++) { minimo[i] = 1023; maximo[i] = 0; }
}

void setup() {
  Serial.begin(115200);
  zerar();
}

void loop() {
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == 't' || c == 'T') modoTexto = !modoTexto;
    if (c == 'r' || c == 'R') zerar();
  }

  int v[5];
  for (uint8_t i = 0; i < 5; i++) {
    v[i] = lerSensor(i);
    if (v[i] < minimo[i]) minimo[i] = v[i];
    if (v[i] > maximo[i]) maximo[i] = v[i];
```

```
  }

  if ((long)(millis() - proximoPrint) < 0) return;

  if (!modoTexto) {
    // Serial Plotter: nome:valor,nome:valor  (20 vezes por segundo)
    proximoPrint = millis() + 50;
    for (uint8_t i = 0; i < 5; i++) {
      Serial.print(NOME_DEDO[i]);
      Serial.print(':');
      Serial.print(v[i]);
      Serial.print(i < 4 ? ',' : '\n');
    }
  } else {
    // Resumo em texto (1 vez por segundo)
    proximoPrint = millis() + 1000;
    for (uint8_t i = 0; i < 5; i++) {
      int variacao = maximo[i] - minimo[i];
      Serial.print(NOME_DEDO[i]);
      Serial.print(F(": agora="));
      Serial.print(v[i]);
      Serial.print(F(" min="));
      Serial.print(minimo[i]);
      Serial.print(F(" max="));
      Serial.print(maximo[i]);
      Serial.print(F(" variacao="));
      Serial.print(variacao);
      if (v[i] < 20 || v[i] > 1003)     Serial.println(F("  FIO SOLTO?"));
      else if (variacao ≥ VARIACAO_BOA) Serial.println(F("  boa"));
      else                              Serial.println(F("  pouca: aproxime o ima"));
    }
    Serial.println();
  }
}
```

Como descobrir o polo do ímã

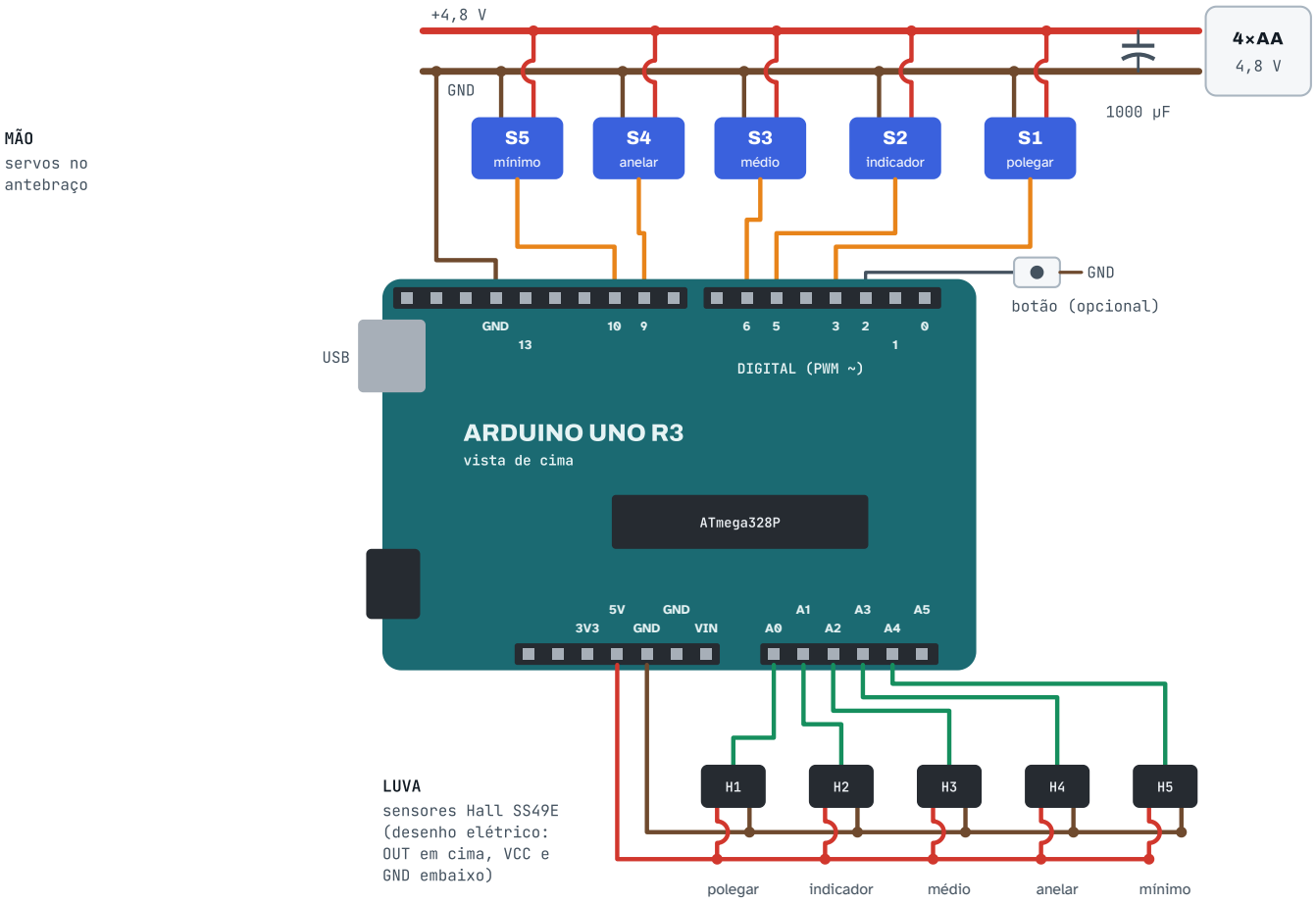
Com o SS49E ligado e o código rodando, aproxime o ímã. Se o valor **sobe**, aquela face é o sul. Se **desce**, é o norte. Marque com caneta. O código calibra sozinho e aceita qualquer polo. O importante é que cada ímã fique sempre na mesma posição.

Esta posição precisa do **SS49E**. O A3144 só diz “perto” ou “longe” e não acompanha o movimento.

VERSÃO 1

Ligação completa com fio

Esta é a versão que você precisa ter funcionando primeiro. Os sensores ligam direto nas entradas analógicas do Uno.



Versão 1 completa. O botão no D2 é opcional, mas recomendo: com ele você calibra e liga a demonstração sem precisar do computador. Em cima ficam os servos alimentados pelas pilhas. No meio está o Uno, com os sinais laranja nos pinos 3, 5, 6, 9 e 10. Embaixo estão os sensores da luva, alimentados pelo 5V do Uno, com as saídas verdes em A0 a A4. Os sensores puxam pouquíssima corrente, então podem ficar no 5V do Uno sem problema. Os servos não podem.

DEDO	SENSOR (SAÍDA)	SERVO (SINAL LARANJA)	SERVO VERMELHO / MARROM
Polegar	H1 → `A0`	S1 → `D3`	linha +4,8 V / linha GND
Indicador	H2 → `A1`	S2 → `D5`	linha +4,8 V / linha GND
Médio	H3 → `A2`	S3 → `D6`	linha +4,8 V / linha GND
Anelar	H4 → `A3`	S4 → `D9`	linha +4,8 V / linha GND
Mínimo	H5 → `A4`	S5 → `D10`	linha +4,8 V / linha GND
Botão (opcional)	Uma perna no `D2`, a outra num `GND`. Sem resistor. Aperto curto = calibrar. Segurar 2 s = modo demonstração.		

Sensores: todos os VCC no `5V` do Uno, todos os GND no `GND` do Uno. **GND comum:** um jumper do `GND` do Uno até a linha GND das pilhas.

O código da versão 1

Grave os sketches na ordem abaixo. Abra o Monitor Serial em **115200** para ver as mensagens.

O que o código faz, 50 vezes por segundo

1. **Lê** os 5 sensores. Cada leitura é a média de 4, para tirar ruído.
2. **Confere o fio:** o SS49E nunca chega perto de 0 V nem de 5 V. Se a leitura chegar lá, é fio solto, e aquele dedo abre e aparece um aviso.
3. **Converte** em fechamento de 0 a 100 % com duas retas: aberto → meio vira 0–50 %, meio → fechado vira 50–100 %. Isso “endireita” a curva do ímã, que muda mais rápido no começo da dobra.
4. **Suaviza** com uma média móvel (`ALFA`), para o dedo não tremer.
5. **Converte em ângulo** entre `ANG_ABERTO` e `ANG_FECHADO` daquele dedo e move o servo aos poucos, no máximo `PASSO_MAX` graus por ciclo. Isso suaviza o movimento e evita pico de corrente.

Calibração em 3 posições (SS49E)

1. LED do pino 13 piscando **devagar**: mão aberta, dedos esticados.
2. Piscando **médio**: dobre **só a base** dos dedos até a metade, como a letra C ou segurando uma lata.
3. Piscando **rápido**: punho fechado.
4. Apagado: pronto. O Monitor Serial mostra se cada dedo ficou “OK”.

A calibração fica **guardada na memória do Uno** (EEPROM). Se ele reiniciar no meio da apresentação, volta funcionando sem calibrar. Mesmo assim, **calibre de novo toda vez que vestir a luva**, com um aperto curto no botão ou digitando `c` .

Modo demonstração

Segure o botão por 2 s (ou digite `d`): a mão faz uma sequência de gestos sozinha, sem a luva. A sequência é mão aberta, punho, conta de 1 a 5, joinha, rock, pinça e uma “onda” de dedo em dedo, e dura uns 18 s em loop.

Serve para duas coisas. Antes da luva ficar pronta, você testa a mecânica e a calibração dos ângulos. No dia, é o plano B se a luva falhar. Os gestos ficam na tabela `GESTOS` do código, e você pode inventar os seus.

Comandos no Monitor Serial (115200)

- `c` : calibrar de novo
- `d` : liga/desliga a demonstração
- `p` : liga/desliga o modo **Serial Plotter**, que mostra uma curva por dedo (0 a 100 %). É ótimo para mostrar à banca.
- `?` : ajuda

Abrir o Monitor Serial reinicia o Uno. É normal, e a calibração guardada volta sozinha.

AJUSTE NO CÓDIGO	PADRÃO	QUANDO MEXER
`ANG_ABERTO` / `ANG_FECHADO`	20 / 160	**Sempre.** Troque pelos ângulos que você anotar no `teste_servos`, um por dedo.
`USAR_A3144`	0	Mude para 1 se usar o A3144 no lugar do SS49E (aí não há calibração).
`CALIBRAR_MEIO`	1	Mude para 0 se preferir calibrar só aberto e fechado. Fica mais rápido, mas o meio do movimento fica menos fiel.
`PASSO_MAX`	6°	Diminua se o Uno reiniciar quando todos os dedos mexem juntos. Aumente se quiser o dedo mais rápido.
`ALFA`	0,3	Diminua (0,15) se o dedo tremer. Aumente (0,5) se ele parecer atrasado.
`ZONA_MORTA`	2°	Aumente para 3 ou 4 se o servo ficar “zumbindo” parado.
`DIFERENCA_MINIMA`	30	Diminua só se um dedo funcionar, mas a calibração rejeitar.
`GESTOS`	19 gestos	Para criar ou mudar gestos da demonstração: % de cada dedo e tempo em ms.

`teste_servos/teste_servos.ino`

Arduino Uno · 189 linhas

Passo 1. Move um servo por vez pelo Monitor Serial para você descobrir os ângulos de aberto e fechado de cada dedo.

```
/*
  ============================================================================
  TESTE DOS SERVOS  -  ferramenta de calibracao (Arduino Uno)
  ============================================================================
  Para que serve:
    Descobrir, dedo por dedo, qual angulo deixa o dedo ABERTO (esticado)
    e qual angulo deixa o dedo FECHADO (dobrado). Esses numeros vao para
    as tabelas ANG_ABERTO e ANG_FECHADO dos programas "mao_com_fio" e
    "mao_sem_fio".

  Como usar:
    1. Grave este programa no Uno.
    2. Abra o Monitor Serial (lupa no canto da IDE) em 115200 baud.
    3. Digite comandos e aperte Enter:
```

```
        2 90    -> move o servo 2 (indicador) para 90 graus
        a       -> leva todos os dedos para ANG_ABERTO (repouso)
        f       -> leva todos os dedos para ANG_FECHADO
        ?       -> mostra a ajuda de novo
    4. Va mudando o angulo aos poucos (de 5 em 5 graus). Quando o dedo
       ficar bem esticado, anote: esse e o ANG_ABERTO. Quando ficar bem
       dobrado SEM o servo forcar (sem zumbido alto), anote: ANG_FECHADO.

  Numeracao dos servos (igual em todos os programas):
    1 = polegar  -> pino D3
    2 = indicador-> pino D5
    3 = medio    -> pino D6
    4 = anelar   -> pino D9
    5 = minimo   -> pino D10

  Ligacao: fio laranja/amarelo do servo = sinal (pino acima),
           vermelho = + do pack de pilhas (4,8 V), marrom = GND.
           O GND do pack TEM que estar ligado no GND do Uno.

  Dica de montagem: grave este programa ANTES de prender o "chifre"
  (a pecinha branca) no servo. Ao ligar, todos vao para 90 graus; prenda
  o chifre com o servo nessa posicao (meio do curso).
  ============================================================================
*/

#include <Servo.h>

// ---------------- Tabelas iguais as dos outros programas ----------------
// Ordem: polegar, indicador, medio, anelar, minimo
const uint8_t PINO_SERVO[5]  = { 3, 5, 6, 9, 10 };
const char*   NOME_DEDO[5]   = { "polegar", "indicador", "medio", "anelar", "minimo" };

// Troque estes numeros pelos que voce anotar no teste.
int ANG_ABERTO[5]  = { 20, 20, 20, 20, 20 };     // dedo esticado
int ANG_FECHADO[5] = { 160, 160, 160, 160, 160 }; // dedo dobrado

const int ANG_INICIAL = 90;        // posicao ao ligar (meio do curso)
const int ATRASO_ENTRE_SERVOS = 300; // ms entre um servo e o proximo

Servo servo[5];
int angAtual[5];

char linha[24];        // texto digitado
uint8_t tamLinha = 0;
unsigned long ultimoCaractere = 0;

// Move os 5 servos, UM DE CADA VEZ, para os angulos da tabela.
// Mover todos juntos puxa muita corrente de uma vez e pode reiniciar o Uno.
void moverTodos(const int angulos[5]) {
  for (uint8_t i = 0; i < 5; i++) {
    servo[i].write(angulos[i]);
    angAtual[i] = angulos[i];
    delay(ATRASO_ENTRE_SERVOS);
  }
}

void mostrarAjuda() {
  Serial.println();
  Serial.println(F("=== TESTE DOS SERVOS - Mao Robotica ==="));
  Serial.println(F("Comandos (digite e aperte Enter):"));
  Serial.println(F("  N ANGULO  -> ex.: 2 90  (servo 1 a 5, angulo 0 a 180)"));
  Serial.println(F("  a         -> todos para ANG_ABERTO (repouso)"));
  Serial.println(F("  f         -> todos para ANG_FECHADO (use depois de ajustar a tabela)"));
  Serial.println(F("  ?         -> esta ajuda"));
  Serial.println(F("Servos: 1=polegar(D3) 2=indicador(D5) 3=medio(D6) 4=anelar(D9) 5=minimo(D10)"));
  Serial.println(F("Va de 5 em 5 graus. Se o servo zumbir forte, volte um pouco!"));
  Serial.println();
}
```

```
void mostrarAngulos() {
  Serial.print(F("Angulos agora: "));
  for (uint8_t i = 0; i < 5; i++) {
    Serial.print(NOME_DEDO[i]);
    Serial.print('=');
    Serial.print(angAtual[i]);
    Serial.print(i < 4 ? F("  ") : F("\n"));
  }
}

// Interpreta o que foi digitado
void executarComando(char* txt) {
  // pula espacos no comeco
  while (*txt == ' ') txt++;
  if (*txt == '\0') return;

  if (*txt == '?' || *txt == 'h' || *txt == 'H') {
    mostrarAjuda();
    return;
  }
  if (*txt == 'a' || *txt == 'A') {
    Serial.println(F("Abrindo todos (ANG_ABERTO)..."));
    moverTodos(ANG_ABERTO);
    mostrarAngulos();
    return;
  }
  if (*txt == 'f' || *txt == 'F') {
    Serial.println(F("Fechando todos (ANG_FECHADO)..."));
    moverTodos(ANG_FECHADO);
    mostrarAngulos();
    return;
  }

  // Formato "N ANGULO"
  char* resto;
  long n = strtol(txt, &resto, 10);
  if (resto == txt) {
    Serial.println(F("Nao entendi. Digite ? para ajuda."));
    return;
  }
  char* fim;
  long ang = strtol(resto, &fim, 10);
  if (fim == resto) {
    Serial.println(F("Falta o angulo. Exemplo: 2 90"));
    return;
  }
  if (n < 1 || n > 5) {
    Serial.println(F("Servo invalido: use 1 a 5."));
    return;
  }
  if (ang < 0 || ang > 180) {
    Serial.println(F("Angulo invalido: use 0 a 180."));
    return;
  }

  uint8_t i = n - 1;
  servo[i].write((int)ang);
  angAtual[i] = ang;
  Serial.print(F("Servo "));
  Serial.print(n);
  Serial.print(F(" ("));
  Serial.print(NOME_DEDO[i]);
  Serial.print(F(") -> "));
  Serial.print(ang);
  Serial.println(F(" graus"));
  mostrarAngulos();
}
```

```
void setup() {
  Serial.begin(115200);

  // Liga os servos um por um, ja em 90 graus, com uma pausa entre eles.
  // (write antes de attach = o primeiro pulso ja sai no angulo certo)
  for (uint8_t i = 0; i < 5; i++) {
    servo[i].write(ANG_INICIAL);
    servo[i].attach(PINO_SERVO[i]);
    angAtual[i] = ANG_INICIAL;
    delay(ATRASO_ENTRE_SERVOS);
  }

  mostrarAjuda();
  mostrarAngulos();
}

void loop() {
  // Junta as letras digitadas ate chegar o Enter.
  while (Serial.available() > 0) {
    char c = Serial.read();
    ultimoCaractere = millis();
    if (c == '\n' || c == '\r') {
      linha[tamLinha] = '\0';
      executarComando(linha);
      tamLinha = 0;
    } else if (tamLinha < sizeof(linha) - 1) {
      linha[tamLinha++] = c;
    }
  }

  // Se o Monitor estiver em "Nenhum final de linha", nao chega Enter.
  // Entao, depois de 200 ms sem letras novas, executa assim mesmo.
  if (tamLinha > 0 && millis() - ultimoCaractere > 200) {
    linha[tamLinha] = '\0';
    executarComando(linha);
    tamLinha = 0;
  }
}
```

COMO ACHAR OS ÂNGULOS DE CADA DEDO

1. Grave o `teste_servos` **antes** de prender o braço do servo. Todos vão para 90°. Prenda o braço nessa posição.
2. Com a mão montada, digite no Monitor Serial `2 80`, `2 70`, `2 60`... até o indicador ficar bem esticado. Anote: esse é o `ANG_ABERTO` dele.
3. Agora suba: `2 100`, `2 110`... até o dedo ficar bem dobrado **sem o servo zumbir forte**. Volte 5° e anote: esse é o `ANG_FECHADO`.
4. Repita para os 5 dedos e copie os números para `mao_com_fio` e `mao_sem_fio`, na mesma ordem: polegar, indicador, médio, anelar, mínimo.

Num dedo, o “aberto” pode ser um número maior que o “fechado”, dependendo do lado em que a linha foi presa. Isso é normal e o código aceita.

mao_com_fio/mao_com_fio.ino

Arduino Uno · versão 1 · 524 linhas

Passo 2. Lê os 5 sensores em A0 a A4, calibra em 3 posições, guarda a calibração na memória e move os servos. Tem modo demonstração e saída para o Serial Plotter.

```
/*
  ============================================================================
  MAO ROBOTICA - VERSAO 1 (COM FIO)  -  Arduino Uno
  ============================================================================
  O que faz:
    Le 5 sensores Hall da luva (um por dedo). Cada sensor fica nas costas
    da mao, logo antes do no do dedo, e o ima fica na falange logo depois.
    Quando o dedo dobra, o ima gira e se afasta, e o dedo da mao robotica
    fecha na mesma proporcao. A calibracao funciona com o sinal subindo ou
    descendo, entao nao importa o lado do ima.

  Ligacoes:
    Sensores Hall (SS49E):  VCC -> 5V do Uno | GND -> GND | OUT -> pino abaixo
      polegar = A0, indicador = A1, medio = A2, anelar = A3, minimo = A4
    Servos SG90 (sinal):    polegar = D3, indicador = D5, medio = D6,
                            anelar = D9, minimo = D10
    Botao (OPCIONAL):       uma perna no D2, a outra no GND. Sem resistor.
    Alimentacao dos servos: pack 4 pilhas AA recarregaveis (4,8 V).
                            + do pack -> fio vermelho dos servos
                            - do pack -> fio marrom dos servos E GND do Uno
    O Uno fica ligado na USB do computador (ou num power bank).
    ATENCAO: os sensores usam o 5V do UNO (nao o pack). Nunca ligue o +
    do pack no pino 5V do Uno: so o GND (-) e compartilhado.

  CALIBRACAO (so no modo SS49E). Siga o LED do pino 13:
    1) piscando DEVAGAR : mao ABERTA, dedos esticados (~3 s)
    2) piscando MEDIO   : dedos dobrados SO NA BASE, ate a metade,
                          como a letra "C" ou segurando uma lata (~3 s)
    3) piscando RAPIDO  : PUNHO fechado (~3 s)
    4) apagado          : pronto!
    A calibracao fica GUARDADA na memoria (EEPROM): ao ligar de novo, o Uno
    usa a ultima. Para calibrar de novo: aperte o botao rapidinho, ou
    digite c no Monitor Serial, ou ligue o Uno com o botao apertado.
    Calibre de novo sempre que vestir a luva.

  MODO DEMONSTRACAO (mao faz gestos sozinha, sem a luva):
    segure o botao por 2 s, ou digite d no Monitor Serial. Repita para sair.
    Use para testar a mecanica e como plano B na apresentacao.

  Monitor Serial em 115200. Digite ? para ver os comandos.
  Serial Plotter: digite p para ver a curva de cada dedo (0 a 100 %).
  ============================================================================
*/

#include <Servo.h>
#include <EEPROM.h>

// ---------------- Tipo de sensor ----------------
// 0 = SS49E (analogico, recomendado)   1 = A3144 (digital, liga/desliga)
// A3144: VCC -> 5V, GND -> GND, OUT -> A0..A4. Usa o pull-up interno.
// Ele so liga com o polo SUL do ima virado para a face com escrita.
#define USAR_A3144 0

// 1 = calibracao em 3 posicoes (aberta, meio, fechada): deixa o movimento
//     mais "reto", porque o sinal do ima muda mais rapido no comeco da dobra.
// 0 = so aberta e fechada (mais rapido de calibrar).
#define CALIBRAR_MEIO 1

// ---------------- Pinos (ordem: polegar, indicador, medio, anelar, minimo)
const uint8_t PINO_SENSOR[5] = { A0, A1, A2, A3, A4 };
const uint8_t PINO_SERVO[5]  = { 3, 5, 6, 9, 10 };
const char*   NOME_DEDO[5]   = { "polegar", "indicador", "medio", "anelar", "minimo" };
const uint8_t PINO_LED   = 13;
```

```
const uint8_t PINO_BOTAO = 2;   // opcional: botao entre D2 e GND

// ---------------- Calibracao dos servos ----------------
// Use o programa "teste_servos" para descobrir estes numeros.
// Sempre entre 0 e 180. ABERTO pode ser MAIOR que FECHADO (servo montado
// ao contrario), o programa aceita os dois jeitos.
// Copie os MESMOS numeros para o "mao_sem_fio" se for usar a versao 2.
int ANG_ABERTO[5]  = { 20, 20, 20, 20, 20 };     // dedo esticado
int ANG_FECHADO[5] = { 160, 160, 160, 160, 160 }; // dedo dobrado

// ---------------- Ajustes finos ----------------
const unsigned long PERIODO_MS = 20;  // 50 vezes por segundo
const int   ZONA_MORTA = 2;           // graus: mudancas menores sao ignoradas (evita tremedeira)
const int   PASSO_MAX  = 6;           // graus por ciclo: limita a velocidade (e o pico de corrente)
const float ALFA = 0.3;               // filtro: perto de 0 = mais suave/lento, 1 = sem filtro
const unsigned long TEMPO_CALIBRACAO_MS = 3000; // tempo de cada etapa (use 2000 ou mais: o 1o segundo e ignorado)
const int   DIFERENCA_MINIMA = 30;    // leitura minima de diferenca entre aberto e fechado (nunca 0)
const float LIMIAR_FECHAR = 70;       // so modo A3144: fecha quando o filtro passa de 70 %
const float LIMIAR_ABRIR  = 30;       // so modo A3144: abre quando cai abaixo de 30 % (histerese)
const unsigned long APERTO_LONGO_MS = 2000; // segurar o botao por 2 s = modo demonstracao
// O SS49E nunca chega perto de 0 V nem de 5 V. Leitura nas pontas = fio solto.
const int   LEITURA_MIN_VALIDA = 20;
const int   LEITURA_MAX_VALIDA = 1003;

// ---------------- Gestos do modo demonstracao ----------------
// Fechamento de cada dedo em % (polegar, indicador, medio, anelar, minimo)
// e quanto tempo o gesto fica parado. Pode inventar os seus!
struct Gesto {
  uint8_t pct[5];
  uint16_t ms;
};
const Gesto GESTOS[] = {
  { {   0,   0,   0,   0,   0 }, 1500 },  // mao aberta
  { { 100, 100, 100, 100, 100 }, 1500 },  // punho
  { { 100,   0, 100, 100, 100 }, 1000 },  // conta 1
  { { 100,   0,   0, 100, 100 }, 1000 },  // conta 2
  { { 100,   0,   0,   0, 100 }, 1000 },  // conta 3
  { { 100,   0,   0,   0,   0 }, 1000 },  // conta 4
  { {   0,   0,   0,   0,   0 }, 1200 },  // conta 5
  { {   0, 100, 100, 100, 100 }, 1800 },  // joinha
  { { 100,   0, 100, 100,   0 }, 1800 },  // rock
  { {  70,  70,   0,   0,   0 }, 1800 },  // pinca / "ok"
  { {   0,   0,   0,   0, 100 },  500 },  // onda: fecha do minimo ao polegar...
  { {   0,   0,   0, 100, 100 },  500 },
  { {   0,   0, 100, 100, 100 },  500 },
  { {   0, 100, 100, 100, 100 },  500 },
  { { 100, 100, 100, 100, 100 },  800 },
  { { 100, 100, 100, 100,   0 },  500 },  // ...e abre na mesma ordem
  { { 100, 100, 100,   0,   0 },  500 },
  { { 100, 100,   0,   0,   0 },  500 },
  { { 100,   0,   0,   0,   0 },  500 },
};
const uint8_t NUM_GESTOS = sizeof(GESTOS) / sizeof(GESTOS[0]);

// ---------------- Calibracao guardada na EEPROM ----------------
const uint16_t ASSINATURA = 0x4D32;   // "M2": muda se o formato mudar
struct Calibracao {
  uint16_t assinatura;
  int aberto[5];
  int meio[5];
  int fechado[5];
  uint8_t conferencia;                // soma dos bytes, para saber se esta inteira
};

// ---------------- Variaveis ----------------
Servo servo[5];
int   angAtual[5];      // angulo que o servo esta agora
int   alvo[5];          // angulo para onde ele quer ir
```

```
int   leituraCrua[5];   // ultimo valor lido do sensor
int   valorAberto[5];   // leitura com a mao aberta (calibracao)
int   valorMeio[5];     // leitura com os dedos ate a metade (calibracao)
int   valorFechado[5];  // leitura com o punho fechado (calibracao)
bool  sensorOk[5];      // false = sensor quase nao mudou na calibracao
bool  meioOk[5];        // false = ponto do meio nao serve, usa so aberto/fechado
bool  fioSolto[5];      // leitura impossivel: fio solto ou sensor queimado
float pctFiltrado[5];   // fechamento filtrado, 0 a 100 %
bool  dedoFechado[5];   // so modo A3144
int   pctFinal[5];      // fechamento usado nos servos, 0 a 100 %

bool  modoDemo = false;
uint8_t gestoAtual = 0;
unsigned long inicioGesto = 0;
bool  modoPlotter = false;

bool  botaoAntes = false;
unsigned long inicioAperto = 0;
bool  apertoLongoFeito = false;

unsigned long proximoCiclo = 0;
unsigned long proximoPrint = 0;
unsigned long proximoAvisoFio = 0;

// ======================== SENSORES ========================

void atualizarServo(uint8_t i, int pct);   // fica la embaixo, na parte dos servos

// Le um sensor. SS49E: media de 4 leituras (0..1023). A3144: 1 = ima perto.
int lerSensor(uint8_t i) {
#if USAR_A3144
  return (digitalRead(PINO_SENSOR[i]) == LOW) ? 1 : 0;  // A3144 puxa para LOW com ima
#else
  int soma = 0;
  for (uint8_t k = 0; k < 4; k++) soma += analogRead(PINO_SENSOR[i]);
  return soma / 4;
#endif
}

bool leituraValida(int leitura) {
  return leitura >= LEITURA_MIN_VALIDA && leitura <= LEITURA_MAX_VALIDA;
}

// Mede a media dos 5 sensores durante TEMPO_CALIBRACAO_MS, piscando o LED.
// O primeiro 1 segundo e ignorado (tempo para voce posicionar a mao).
void medirMedia(int resultado[5], unsigned long meioPiscar) {
  long soma[5] = { 0, 0, 0, 0, 0 };
  long n = 0;
  unsigned long inicio = millis();
  unsigned long ultimoPisca = inicio;
  bool led = false;
  while (millis() - inicio < TEMPO_CALIBRACAO_MS) {
    if (millis() - ultimoPisca >= meioPiscar) {
      ultimoPisca = millis();
      led = !led;
      digitalWrite(PINO_LED, led);
    }
    if (millis() - inicio > 1000) {
      for (uint8_t i = 0; i < 5; i++) soma[i] += lerSensor(i);
      n++;
    }
    delay(5);
  }
  if (n == 0) n = 1;  // protecao: TEMPO_CALIBRACAO_MS curto demais (1 s ou menos)
  for (uint8_t i = 0; i < 5; i++) resultado[i] = soma[i] / n;
  digitalWrite(PINO_LED, LOW);
}
```

```
// Confere cada dedo depois de calibrar (ou de carregar da EEPROM).
void conferirCalibracao(bool mostrar) {
  for (uint8_t i = 0; i < 5; i++) {
    int diferenca = abs(valorFechado[i] - valorAberto[i]);
    bool valida = leituraValida(valorAberto[i]) && leituraValida(valorFechado[i]);
    sensorOk[i] = valida && diferenca ⩾ DIFERENCA_MINIMA && diferenca > 0;  // > 0: evita dividir por zero

    // O meio tem que ficar ENTRE aberto e fechado, com folga dos dois lados.
    long lado = (valorFechado[i] > valorAberto[i]) ? 1 : -1;
    long m = (long)(valorMeio[i] - valorAberto[i]) * lado;
    long f = (long)(valorFechado[i] - valorAberto[i]) * lado;
    long folga = f / 8;
    meioOk[i] = CALIBRAR_MEIO && sensorOk[i] && m > folga && m < f - folga;

    if (!mostrar) continue;
    Serial.print(F("  "));
    Serial.print(NOME_DEDO[i]);
    Serial.print(F(": aberto="));
    Serial.print(valorAberto[i]);
#if CALIBRAR_MEIO
    Serial.print(F(" meio="));
    Serial.print(valorMeio[i]);
#endif
    Serial.print(F(" fechado="));
    Serial.print(valorFechado[i]);
    if (!valida) {
      Serial.println(F("  FIO SOLTO? Leitura impossivel. Confira VCC, GND e OUT deste sensor."));
    } else if (!sensorOk[i]) {
      Serial.println(F("  POUCA DIFERENCA! Aproxime o ima do sensor. Este dedo vai ficar parado."));
    } else if (CALIBRAR_MEIO && !meioOk[i]) {
      Serial.println(F("  OK (o meio nao ficou entre aberto e fechado: usando so os dois)"));
    } else {
      Serial.println(F("  OK"));
    }
  }
}

uint8_t somaBytes(const Calibracao& c) {
  const uint8_t* p = (const uint8_t*)&c;
  uint8_t s = 0;
  for (uint8_t k = 0; k < offsetof(Calibracao, conferencia); k++) s += p[k];
  return s;
}

void salvarCalibracao() {
  Calibracao c;
  c.assinatura = ASSINATURA;
  for (uint8_t i = 0; i < 5; i++) {
    c.aberto[i] = valorAberto[i];
    c.meio[i] = valorMeio[i];
    c.fechado[i] = valorFechado[i];
  }
  c.conferencia = somaBytes(c);
  EEPROM.put(0, c);   // put so grava os bytes que mudaram (poupa a EEPROM)
}

bool carregarCalibracao() {
  Calibracao c;
  EEPROM.get(0, c);
  if (c.assinatura ≠ ASSINATURA || c.conferencia ≠ somaBytes(c)) return false;
  for (uint8_t i = 0; i < 5; i++) {
    valorAberto[i] = c.aberto[i];
    valorMeio[i] = c.meio[i];
    valorFechado[i] = c.fechado[i];
  }
  return true;
}
```

```
// Abre a mao devagar (respeitando PASSO_MAX) e espera chegar.
void abrirMao() {
  for (uint8_t t = 0; t < 40; t++) {
    for (uint8_t i = 0; i < 5; i++) atualizarServo(i, 0);
    delay(PERIODO_MS);
  }
  for (uint8_t i = 0; i < 5; i++) pctFinal[i] = 0;
}

void calibrarSensores() {
  // Enquanto calibra, a mao robotica fica aberta e parada.
  abrirMao();
#if CALIBRAR_MEIO
  Serial.println(F("CALIBRACAO 1/3: mao ABERTA, dedos esticados (LED devagar)..."));
  medirMedia(valorAberto, 400);
  Serial.println(F("CALIBRACAO 2/3: dobre so a BASE dos dedos ate a METADE, como um C (LED medio)..."));
  medirMedia(valorMeio, 180);
  Serial.println(F("CALIBRACAO 3/3: FECHE o punho (LED rapido)..."));
  medirMedia(valorFechado, 70);
#else
  Serial.println(F("CALIBRACAO 1/2: mao ABERTA, dedos esticados (LED devagar)..."));
  medirMedia(valorAberto, 400);
  Serial.println(F("CALIBRACAO 2/2: FECHE o punho (LED rapido)..."));
  medirMedia(valorFechado, 70);
  for (uint8_t i = 0; i < 5; i++) valorMeio[i] = (valorAberto[i] + valorFechado[i]) / 2;
#endif
  Serial.println(F("Resultado:"));
  conferirCalibracao(true);
  salvarCalibracao();
  Serial.println(F("Calibracao guardada na memoria."));
  for (uint8_t i = 0; i < 5; i++) pctFiltrado[i] = 0;
}

// Converte a leitura do sensor em fechamento de 0 a 100 %.
// Funciona com o ima de qualquer lado (a leitura pode subir ou descer).
// Com o ponto do meio, sao duas retas: aberto→meio = 0..50 % e
// meio→fechado = 50..100 %. Isso corrige a curva do ima.
int paraPorcento(uint8_t i, int leitura) {
  if (!sensorOk[i]) return 0;
  long lado = (valorFechado[i] > valorAberto[i]) ? 1 : -1;
  long v = (long)(leitura - valorAberto[i]) * lado;          // 0 = aberto
  long f = (long)(valorFechado[i] - valorAberto[i]) * lado;  // > 0
  long p;
  if (meioOk[i]) {
    long m = (long)(valorMeio[i] - valorAberto[i]) * lado;   // entre 0 e f
    if (v <= m) p = v * 50L / m;
    else        p = 50L + (v - m) * 50L / (f - m);
  } else {
    p = v * 100L / f;
  }
  return constrain(p, 0, 100);
}

// ======================== SERVOS ========================

// Leva o servo i na direcao do fechamento pct (0..100 %),
// com zona morta e limite de velocidade.
void atualizarServo(uint8_t i, int pct) {
  int alvoNovo = map(pct, 0, 100, ANG_ABERTO[i], ANG_FECHADO[i]);
  bool extremo = (pct == 0 || pct == 100);      // nos extremos vai ate o fim
  if (abs(alvoNovo - alvo[i]) > ZONA_MORTA || extremo) {
    alvo[i] = alvoNovo;
  }
  int falta = alvo[i] - angAtual[i];
  if (falta != 0) {
    angAtual[i] += constrain(falta, -PASSO_MAX, PASSO_MAX);
    servo[i].write(angAtual[i]);
  }
```

```
}

// ======================== MODOS E COMANDOS ========================

void ligarDemo(bool ligar) {
  modoDemo = ligar;
  gestoAtual = 0;
  inicioGesto = millis();
  Serial.println(ligar ? F(">> MODO DEMONSTRACAO ligado (a luva e ignorada)")
                       : F(">> MODO DEMONSTRACAO desligado (voltou para a luva)"));
  for (uint8_t i = 0; i < 5; i++) pctFiltrado[i] = 0;
}

void mostrarAjuda() {
  Serial.println(F("Comandos (digite e aperte Enter):"));
  Serial.println(F("  c = calibrar de novo   d = liga/desliga demonstracao"));
  Serial.println(F("  p = liga/desliga modo Serial Plotter   ? = esta ajuda"));
  Serial.println(F("Botao no D2: aperto curto = calibrar | segurar 2 s = demonstracao"));
}

void recalibrar() {
#if USAR_A3144
  Serial.println(F("Modo A3144: nao precisa calibrar."));
#else
  if (modoDemo) ligarDemo(false);
  calibrarSensores();
  proximoCiclo = millis();
#endif
}

void lerComandos() {
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == 'c' || c == 'C') recalibrar();
    else if (c == 'd' || c == 'D') ligarDemo(!modoDemo);
    else if (c == 'p' || c == 'P') {
      modoPlotter = !modoPlotter;
      if (!modoPlotter) Serial.println(F(">> modo texto"));
    }
    else if (c == '?') mostrarAjuda();
  }
}

// Botao entre D2 e GND (INPUT_PULLUP: apertado = LOW).
// Sem botao ligado, o pino fica em HIGH e nada acontece.
void lerBotao() {
  bool apertado = (digitalRead(PINO_BOTAO) == LOW);
  unsigned long agora = millis();
  if (apertado && !botaoAntes) {
    inicioAperto = agora;
    apertoLongoFeito = false;
  }
  if (apertado && !apertoLongoFeito && agora - inicioAperto >= APERTO_LONGO_MS) {
    apertoLongoFeito = true;
    ligarDemo(!modoDemo);
  }
  if (!apertado && botaoAntes && !apertoLongoFeito && agora - inicioAperto >= 40) {
    recalibrar();   // aperto curto (40 ms ignora ruido do contato)
  }
  botaoAntes = apertado;
}

// ======================== SAIDA NO MONITOR ========================

void imprimirDepuracao() {
  if (modoPlotter) {
    // Formato do Serial Plotter da Arduino IDE 2: nome:valor,nome:valor
    for (uint8_t i = 0; i < 5; i++) {
```

```
      Serial.print(NOME_DEDO[i]);
      Serial.print(':');
      Serial.print(pctFinal[i]);
      Serial.print(i < 4 ? ',' : '\n');
    }
    return;
  }
  Serial.print(modoDemo ? F("DEMO ") : F("cru:"));
  if (!modoDemo) {
    for (uint8_t i = 0; i < 5; i++) { Serial.print(' '); Serial.print(leituraCrua[i]); }
  }
  Serial.print(F(" | %:"));
  for (uint8_t i = 0; i < 5; i++) { Serial.print(' '); Serial.print(pctFinal[i]); }
  Serial.print(F(" | ang:"));
  for (uint8_t i = 0; i < 5; i++) { Serial.print(' '); Serial.print(angAtual[i]); }
  Serial.println();
}

// ======================== SETUP E LOOP ========================

void setup() {
  Serial.begin(115200);
  pinMode(PINO_LED, OUTPUT);
  pinMode(PINO_BOTAO, INPUT_PULLUP);
  Serial.println(F("\n=== Mao Robotica - Versao 1 (com fio) ==="));

  // Liga os servos UM DE CADA VEZ, ja na posicao aberta.
  for (uint8_t i = 0; i < 5; i++) {
    servo[i].write(ANG_ABERTO[i]);
    servo[i].attach(PINO_SERVO[i]);
    angAtual[i] = ANG_ABERTO[i];
    alvo[i] = ANG_ABERTO[i];
    pctFiltrado[i] = 0;
    pctFinal[i] = 0;
    dedoFechado[i] = false;
    fioSolto[i] = false;
    delay(300);
  }

#if USAR_A3144
  for (uint8_t i = 0; i < 5; i++) pinMode(PINO_SENSOR[i], INPUT_PULLUP);
  Serial.println(F("Sensor: A3144 (digital). Sem calibracao."));
#else
  Serial.println(F("Sensor: SS49E (analogico)."));
  bool botaoNoBoot = (digitalRead(PINO_BOTAO) == LOW);
  if (!botaoNoBoot && carregarCalibracao()) {
    Serial.println(F("Usando a calibracao guardada:"));
    conferirCalibracao(true);
    Serial.println(F("(Vestiu a luva de novo? Aperte o botao ou digite c para calibrar.)"));
  } else {
    calibrarSensores();
  }
  botaoAntes = (digitalRead(PINO_BOTAO) == LOW);
  apertoLongoFeito = true;   // o aperto usado no boot nao conta como comando
#endif

  mostrarAjuda();
  Serial.println(F("Pronto! Mexa os dedos da luva."));
  proximoCiclo = millis();
}

void loop() {
  lerComandos();
  lerBotao();

  // Tudo acontece a cada PERIODO_MS (20 ms = 50 vezes por segundo)
  if ((long)(millis() - proximoCiclo) < 0) return;
  proximoCiclo += PERIODO_MS;
```

```
  if (modoDemo) {
    if (millis() - inicioGesto >= GESTOS[gestoAtual].ms) {
      gestoAtual = (gestoAtual + 1) % NUM_GESTOS;
      inicioGesto = millis();
    }
    for (uint8_t i = 0; i < 5; i++) {
      pctFinal[i] = GESTOS[gestoAtual].pct[i];
      atualizarServo(i, pctFinal[i]);
    }
  } else {
    for (uint8_t i = 0; i < 5; i++) {
      leituraCrua[i] = lerSensor(i);

#if USAR_A3144
      // Liga/desliga: filtra e usa dois limites (histerese) para nao "piscar"
      float bruto = leituraCrua[i] ? 100.0 : 0.0;
      pctFiltrado[i] += ALFA * (bruto - pctFiltrado[i]);
      if (!dedoFechado[i] && pctFiltrado[i] > LIMIAR_FECHAR) dedoFechado[i] = true;
      if (dedoFechado[i] && pctFiltrado[i] < LIMIAR_ABRIR)   dedoFechado[i] = false;
      pctFinal[i] = dedoFechado[i] ? 100 : 0;
#else
      // Fio solto: o dedo abre (mais seguro) e aparece um aviso.
      fioSolto[i] = !leituraValida(leituraCrua[i]);
      int bruto = fioSolto[i] ? 0 : paraPorcento(i, leituraCrua[i]);
      // Analogico: suaviza (media movel exponencial)
      pctFiltrado[i] += ALFA * (bruto - pctFiltrado[i]);
      pctFinal[i] = (int)(pctFiltrado[i] + 0.5);
#endif

      atualizarServo(i, pctFinal[i]);
    }
  }

  // Mostra os numeros 5 vezes por segundo (20 no modo Plotter)
  if ((long)(millis() - proximoPrint) >= 0) {
    proximoPrint = millis() + (modoPlotter ? 50 : 200);
    imprimirDepuracao();
  }

#if !USAR_A3144
  // Aviso de fio solto a cada 2 s (fora do modo Plotter, para nao sujar o grafico)
  if (!modoDemo && !modoPlotter && (long)(millis() - proximoAvisoFio) >= 0) {
    proximoAvisoFio = millis() + 2000;
    for (uint8_t i = 0; i < 5; i++) {
      if (fioSolto[i]) {
        Serial.print(F("!! FIO SOLTO? sensor do "));
        Serial.println(NOME_DEDO[i]);
      }
    }
  }
#endif
}
```

VERSÃO 2

Sem fio com dois ESP32

Comece aqui só quando a versão 1 estiver funcionando. A parte dos servos e das pilhas não muda nada. Muda só como os dados chegam ao Uno.

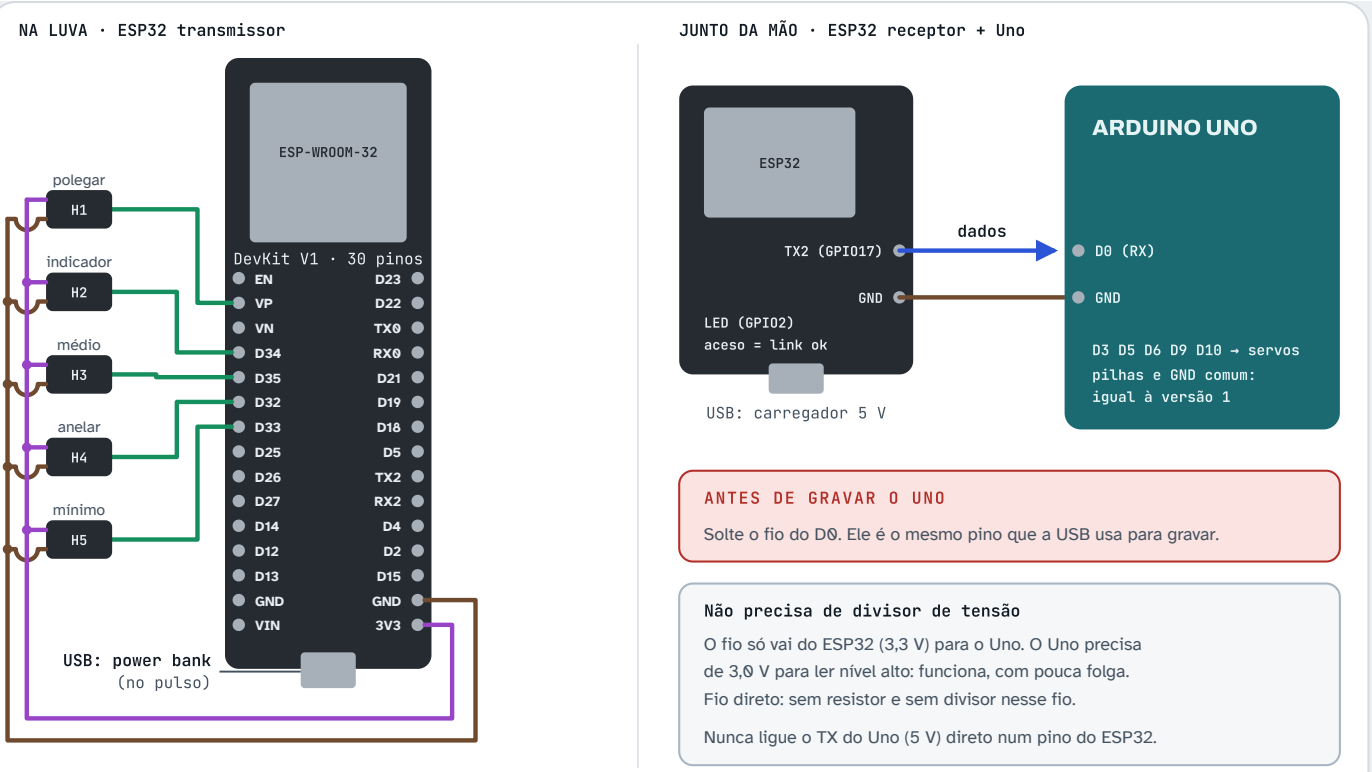
Qual ESP32 comprar

Dois **ESP32 DevKit V1** com módulo **ESP-WROOM-32** (30 pinos), iguais entre si. É o mais comum no Brasil e é para ele que os pinos do guia foram escritos.

- Não compre ESP32-C3, S2 ou S3 para este guia. O ESP-NOW até funciona neles, mas os números dos pinos são outros.
- A versão de 38 pinos também serve. Ligue pelos nomes impressos na placa (D32, D34…), não pela posição.
- Veja se a placa tem o chip USB CP2102 ou CH340, porque você vai precisar do driver correspondente.

Preparar a Arduino IDE

1. Em *Arquivo → Preferências → URLs adicionais*, cole: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
2. Em *Ferramentas → Placa → Gerenciador de placas*, instale **esp32 by Espressif Systems**. O código funciona nas versões 2.x e 3.x.
3. Escolha a placa **DOIT ESP32 DEVKIT V1**, ou *ESP32 Dev Module*.
4. Se ficar parado em “Connecting…”, segure o botão **BOOT** até começar a gravar.
5. Não conecte nenhum dos dois ESP32 a uma rede Wi-Fi. Isso muda o canal do rádio e o ESP-NOW para de funcionar.



Versão 2. À esquerda, a luva: sensores em 3,3 V (fio roxo) e saídas nos pinos VP, D34, D35, D32 e D33, que são todos do ADC1. O ADC2 não funciona quando o rádio está ligado. À direita, o receptor manda os dados pelo TX2 até o D0 do Uno. A ordem dos pinos pode mudar em placas de 38 pinos, então confie no nome impresso na placa e não na posição do desenho.

Como gravar e testar, nesta ordem

1. Grave `receptor_esp32` no ESP32 que fica junto da mão. Abra o Monitor Serial dele em 115200.
2. Grave `luva_esp32` no ESP32 da luva. Siga a calibração pelo LED azul, igual à versão 1: aberta, meio (letra C) e punho. Ela fica guardada. Para calibrar de novo, aperte rapidinho o botão **BOOT** da placa com o programa rodando.
3. O LED do receptor acende quando os pacotes chegam. Desligue a luva: em menos de meio segundo ele apaga.
4. Com o **D0 desconectado**, grave `mao_sem_fio` no Uno. Use os mesmos ângulos que você calibrou na versão 1.
5. Ligue TX2 (GPIO17) do receptor no D0 do Uno e um GND do receptor num GND do Uno.
6. Mexa os dedos. Depois desligue a luva e confira que a mão abre sozinha. Esse é o failsafe.
7. Plano B: no Uno, segure o botão do D2 por 2 s e a mão entra no modo demonstração, mesmo sem rádio.

O que trafega entre as placas

Pelo ar (luva → receptor), 50 vezes por segundo, 8 bytes:

BYTE	CONTEÚDO
0–1	`'M' 'R'`, a assinatura do projeto
2	número de sequência
3–7	fechamento de cada dedo, 0 a 100 %

Pelo fio (receptor → Uno), 7 bytes:

BYTE	CONTEÚDO
0	`0xFF`, o início do quadro
1–5	polegar, indicador, médio, anelar, mínimo (0–100)
6	XOR dos 5 valores, para conferir

Todos os valores são menores que 128, então o `0xFF` nunca aparece no meio dos dados. Se o Uno perder um byte, ele espera o próximo `0xFF` e se acerta sozinho.

`luva_esp32/luva_esp32.ino`

ESP32 DevKit V1 da luva · versão 2 · 357 linhas

Lê os sensores, calibra em 3 posições (guardada na memória; botão BOOT calibra de novo) e envia o fechamento de cada dedo (0 a 100 %) por ESP-NOW.

```
/*
  ============================================================================
  MAO ROBOTICA - VERSAO 2 (SEM FIO) - LUVA  -  ESP32 DevKit V1 (transmissor)
  ============================================================================
  O que faz:
    Le os 5 sensores Hall da luva, calcula quanto cada dedo esta fechado
    (0 a 100 %) e manda pelo ar (ESP-NOW) cerca de 50 vezes por segundo
    para o outro ESP32, que fica junto da mao robotica.

  Ligacoes (SS49E):  VCC -> 3V3 do ESP32 | GND -> GND | OUT -> pino abaixo
      polegar = GPIO36 (no DevKit aparece como "VP"), indicador = GPIO34,
      medio   = GPIO35, anelar = GPIO32, minimo = GPIO33
    (nessa ordem eles ficam em sequencia na fileira de pinos: VP, D34, D35,
     D32, D33; o VN, entre VP e D34, fica sem uso)
    Esses pinos sao do ADC1, que funciona junto com o Wi-Fi.
    (Os pinos do ADC2 NAO funcionam enquanto o Wi-Fi/ESP-NOW esta ligado.)
    !!! Na luva o SS49E vai no 3V3, NUNCA no 5V/VIN: com 5 V a saida dele
        passa de 3,3 V e pode queimar o pino do ESP32. Pelo mesmo motivo,
        NAO use modulos KY-024/KY-003 alimentados com 5 V aqui. !!!

  Este codigo e para o ESP32 "classico" (DevKit V1, modulo ESP-WROOM-32).
  ESP32-S3/C3/C6 tem outros numeros de pino.

  CALIBRACAO (so no modo SS49E). Siga o LED azul (GPIO2):
```

```
    1) piscando DEVAGAR : mao ABERTA, dedos esticados (~3 s)
    2) piscando MEDIO   : dedos dobrados SO NA BASE, ate a metade, como um C
    3) piscando RAPIDO  : PUNHO fechado (~3 s)
    4) depois pisca curtinho de vez em quando = enviando.
    A calibracao fica GUARDADA na memoria do ESP32: ao ligar de novo, ele
    usa a ultima. Para calibrar de novo, aperte o botao BOOT da placa
    (rapidinho, com o programa ja rodando) ou digite c no Monitor Serial.
    Calibre de novo sempre que vestir a luva.
    (Nao segure o BOOT enquanto liga a placa: isso entra no modo gravacao.)

  Placa na IDE: "ESP32 Dev Module" (ou "DOIT ESP32 DEVKIT V1").
  Monitor Serial: 115200.
  ================================================================================
*/

#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>
#include <Preferences.h>

// ---------------- Tipo de sensor ----------------
// 0 = SS49E (analogico, recomendado)   1 = A3144 (digital)
// ATENCAO A3144: precisa de VCC ≥ 4,5 V → ligue o VCC dele no pino VIN
// (5 V da USB). So funciona com o ESP32 alimentado pela USB/power bank;
// com bateria ligada no 3V3 o VIN fica sem 5 V e o A3144 nao liga.
// A saida dele e "coletor aberto": coloque um resistor de
// 10 k entre cada saida e o 3V3. NAO ligue o resistor no 5 V!
// (Os GPIO34, 35 e 36 nao tem pull-up interno, por isso o resistor.)
// Use o A3144 avulso: o modulo KY-003 ja tem resistor para o 5 V.
#define USAR_A3144 0

// 1 = calibracao em 3 posicoes (aberta, meio, fechada): movimento mais "reto".
// 0 = so aberta e fechada.
#define CALIBRAR_MEIO 1

// ---------------- Pinos (ordem: polegar, indicador, medio, anelar, minimo)
const uint8_t PINO_SENSOR[5] = { 36, 34, 35, 32, 33 };
const char*   NOME_DEDO[5]   = { "polegar", "indicador", "medio", "anelar", "minimo" };
const uint8_t PINO_LED = 2;          // LED azul da placa
const uint8_t PINO_BOTAO = 0;        // botao BOOT da placa (ja tem pull-up)

// ---------------- Ajustes ----------------
const uint8_t CANAL_WIFI = 1;        // os dois ESP32 precisam usar o MESMO canal
const unsigned long PERIODO_MS = 20; // 50 envios por segundo
const uint8_t LEITURAS_MEDIA = 8;    // quantas leituras do ADC para fazer a media
const float ALFA = 0.3;              // filtro: perto de 0 = mais suave, 1 = sem filtro
const unsigned long TEMPO_CALIBRACAO_MS = 3000; // use 2000 ou mais (o 1o segundo e ignorado)
const int   DIFERENCA_MINIMA = 120;  // em leituras do ADC (0..4095); nunca 0
// Com 3,3 V o SS49E fica entre ~0,9 e ~2,4 V. Leitura nas pontas = fio solto
// (ou um clone de 49E que nao funciona em 3,3 V).
const int   LEITURA_MIN_VALIDA = 60;
const int   LEITURA_MAX_VALIDA = 4030;
const float LIMIAR_FECHAR = 70;      // so A3144 (histerese)
const float LIMIAR_ABRIR  = 30;

// ---------------- Pacote enviado (IGUAL no receptor!) ----------------
// Outra mao robotica com este programa por perto? Troque MAGIC_2 (ex.: 'X')
// aqui E no receptor_esp32, senao uma luva mexe a mao da outra.
const uint8_t MAGIC_1 = 'M';
const uint8_t MAGIC_2 = 'R';
typedef struct __attribute__((packed)) {
  uint8_t magic[2];   // 'M','R' = "Mao Robotica" (para reconhecer o pacote)
  uint8_t seq;        // numero do pacote (vai de 0 a 255 e volta)
  uint8_t pct[5];     // fechamento de cada dedo, 0..100 %
} PacoteMao;

uint8_t ENDERECO_TODOS[6] = { 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF }; // broadcast
```

```
PacoteMao pacote;
int   leituraCrua[5];
int   valorAberto[5];
int   valorMeio[5];
int   valorFechado[5];
bool  sensorOk[5];
bool  meioOk[5];
bool  fioSolto[5];
Preferences memoria;
bool  botaoAntes = false;
unsigned long inicioAperto = 0;
unsigned long proximoAvisoFio = 0;
float pctFiltrado[5];
bool  dedoFechado[5];
unsigned long proximoEnvio = 0;
unsigned long proximoPrint = 0;
unsigned long errosEnvio = 0;

int lerSensor(uint8_t i) {
#if USAR_A3144
  return (digitalRead(PINO_SENSOR[i]) == LOW) ? 1 : 0;   // LOW = ima perto
#else
  long soma = 0;
  for (uint8_t k = 0; k < LEITURAS_MEDIA; k++) soma += analogRead(PINO_SENSOR[i]);
  return soma / LEITURAS_MEDIA;                          // 0..4095
#endif
}

// Media dos sensores durante TEMPO_CALIBRACAO_MS, piscando o LED.
// O primeiro 1 segundo e ignorado (tempo para posicionar a mao).
void medirMedia(int resultado[5], unsigned long meioPiscar) {
  long soma[5] = { 0, 0, 0, 0, 0 };
  long n = 0;
  unsigned long inicio = millis();
  unsigned long ultimoPisca = inicio;
  bool led = false;
  while (millis() - inicio < TEMPO_CALIBRACAO_MS) {
    if (millis() - ultimoPisca >= meioPiscar) {
      ultimoPisca = millis();
      led = !led;
      digitalWrite(PINO_LED, led);
    }
    if (millis() - inicio > 1000) {
      for (uint8_t i = 0; i < 5; i++) soma[i] += lerSensor(i);
      n++;
    }
    delay(5);
  }
  if (n == 0) n = 1;  // protecao: TEMPO_CALIBRACAO_MS curto demais (no ESP32, dividir por 0 trava a placa)
  for (uint8_t i = 0; i < 5; i++) resultado[i] = soma[i] / n;
  digitalWrite(PINO_LED, LOW);
}

bool leituraValida(int leitura) {
  return leitura >= LEITURA_MIN_VALIDA && leitura <= LEITURA_MAX_VALIDA;
}

// Confere cada dedo depois de calibrar (ou de carregar da memoria).
void conferirCalibracao() {
  for (uint8_t i = 0; i < 5; i++) {
    int diferenca = abs(valorFechado[i] - valorAberto[i]);
    bool valida = leituraValida(valorAberto[i]) && leituraValida(valorFechado[i]);
    sensorOk[i] = valida && diferenca >= DIFERENCA_MINIMA && diferenca > 0;  // > 0: evita dividir por zero
    long lado = (valorFechado[i] > valorAberto[i]) ? 1 : -1;
    long m = (long)(valorMeio[i] - valorAberto[i]) * lado;
    long f = (long)(valorFechado[i] - valorAberto[i]) * lado;
    meioOk[i] = CALIBRAR_MEIO && sensorOk[i] && m > f / 8 && m < f - f / 8;
    const char* situacao = !valida ? "FIO SOLTO? (ou 49E que nao funciona em 3,3 V)"
```

```
                        : !sensorOk[i] ? "POUCA DIFERENCA! Aproxime o ima. Dedo vai ficar parado."
                        : (CALIBRAR_MEIO && !meioOk[i]) ? "OK (meio fora da faixa: usando so aberto/fechado)"
                        : "OK";
    Serial.printf("  %-9s aberto=%4d meio=%4d fechado=%4d  %s\n", NOME_DEDO[i],
                  valorAberto[i], valorMeio[i], valorFechado[i], situacao);
  }
}

void salvarCalibracao() {
  int dados[15];
  for (uint8_t i = 0; i < 5; i++) { dados[i] = valorAberto[i]; dados[5 + i] = valorMeio[i]; dados[10 + i] = valorFechado[i]; }
  memoria.putBytes("cal", dados, sizeof(dados));
}

bool carregarCalibracao() {
  int dados[15];
  if (memoria.getBytes("cal", dados, sizeof(dados)) != sizeof(dados)) return false;
  for (uint8_t i = 0; i < 5; i++) { valorAberto[i] = dados[i]; valorMeio[i] = dados[5 + i]; valorFechado[i] = dados[10 + i]; }
  return true;
}

void calibrarSensores() {
  // Durante a calibracao nada e enviado: o Uno abre a mao (failsafe).
#if CALIBRAR_MEIO
  Serial.println("CALIBRACAO 1/3: mao ABERTA, dedos esticados (LED devagar)...");
  medirMedia(valorAberto, 400);
  Serial.println("CALIBRACAO 2/3: dobre so a BASE dos dedos ate a METADE, como um C (LED medio)...");
  medirMedia(valorMeio, 180);
  Serial.println("CALIBRACAO 3/3: FECHE o punho (LED rapido)...");
  medirMedia(valorFechado, 70);
#else
  Serial.println("CALIBRACAO 1/2: mao ABERTA, dedos esticados (LED devagar)...");
  medirMedia(valorAberto, 400);
  Serial.println("CALIBRACAO 2/2: FECHE o punho (LED rapido)...");
  medirMedia(valorFechado, 70);
  for (uint8_t i = 0; i < 5; i++) valorMeio[i] = (valorAberto[i] + valorFechado[i]) / 2;
#endif
  Serial.println("Resultado:");
  conferirCalibracao();
  salvarCalibracao();
  Serial.println("Calibracao guardada na memoria.");
  for (uint8_t i = 0; i < 5; i++) pctFiltrado[i] = 0;
}

// Leitura -> 0..100 %. Funciona com o ima de qualquer lado.
// Com o ponto do meio sao duas retas (0..50 % e 50..100 %), que corrigem a curva do ima.
int paraPorcento(uint8_t i, int leitura) {
  if (!sensorOk[i]) return 0;
  long lado = (valorFechado[i] > valorAberto[i]) ? 1 : -1;
  long v = (long)(leitura - valorAberto[i]) * lado;
  long f = (long)(valorFechado[i] - valorAberto[i]) * lado;
  long p;
  if (meioOk[i]) {
    long m = (long)(valorMeio[i] - valorAberto[i]) * lado;
    if (v <= m) p = v * 50L / m;
    else        p = 50L + (v - m) * 50L / (f - m);
  } else {
    p = v * 100L / f;
  }
  return constrain(p, 0, 100);
}

// Botao BOOT (GPIO0, apertado = LOW) ou letra c no Monitor: calibra de novo.
bool pediuCalibracao() {
  bool pediu = false;
  while (Serial.available() > 0) {
    char c = Serial.read();
    if (c == 'c' || c == 'C') pediu = true;
```

```
  }
  bool apertado = (digitalRead(PINO_BOTAO) == LOW);
  if (apertado && !botaoAntes) inicioAperto = millis();
  if (!apertado && botaoAntes && millis() - inicioAperto >= 40) pediu = true;  // 40 ms ignora ruido
  botaoAntes = apertado;
  return pediu;
}

void setup() {
  Serial.begin(115200);
  delay(300);
  pinMode(PINO_LED, OUTPUT);
  Serial.println("\n=== Mao Robotica - Luva (ESP32 transmissor) ===");

  // ---- Liga o Wi-Fi so para o ESP-NOW (nao conecta em roteador) ----
  // Fica ANTES da calibracao: com o radio ligado o consumo muda e as
  // leituras do ADC mudam um pouco; calibrar nas mesmas condicoes do uso.
  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  if (esp_wifi_set_channel(CANAL_WIFI, WIFI_SECOND_CHAN_NONE) != ESP_OK) {
    Serial.println("ERRO: nao consegui mudar o canal do Wi-Fi!");
  }
  uint8_t canal = 0;
  wifi_second_chan_t canal2;
  esp_wifi_get_channel(&canal, &canal2);
  Serial.printf("Canal Wi-Fi: %u (tem que ser igual ao do receptor)\n", canal);

  uint8_t mac[6];
  esp_wifi_get_mac(WIFI_IF_STA, mac);
  Serial.printf("MAC desta luva: %02X:%02X:%02X:%02X:%02X:%02X\n",
                mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ERRO: nao consegui iniciar o ESP-NOW. Reiniciando...");
    delay(2000);
    ESP.restart();
  }

  // Cadastra o endereco "broadcast" (todo mundo) como destino
  esp_now_peer_info_t destino = {};
  memcpy(destino.peer_addr, ENDERECO_TODOS, 6);
  destino.channel = 0;              // 0 = usa o canal atual
  destino.ifidx = WIFI_IF_STA;
  destino.encrypt = false;
  if (esp_now_add_peer(&destino) != ESP_OK) {
    Serial.println("ERRO: nao consegui adicionar o destino broadcast.");
  }

  pacote.magic[0] = MAGIC_1;
  pacote.magic[1] = MAGIC_2;
  pacote.seq = 0;
  for (uint8_t i = 0; i < 5; i++) {
    pacote.pct[i] = 0;
    pctFiltrado[i] = 0;
    dedoFechado[i] = false;
  }

#if USAR_A3144
  for (uint8_t i = 0; i < 5; i++) pinMode(PINO_SENSOR[i], INPUT);  // pull-up externo 10k no 3V3
  Serial.println("Sensor: A3144 (digital). Sem calibracao.");
#else
  analogReadResolution(12);           // 0..4095
  analogSetAttenuation(ADC_11db);     // le de 0 ate ~3,1 V
  Serial.println("Sensor: SS49E (analogico).");
  memoria.begin("mao", false);
  if (carregarCalibracao()) {
    Serial.println("Usando a calibracao guardada:");
    conferirCalibracao();
```

```
    Serial.println("(Vestiu a luva de novo? Aperte BOOT ou digite c para calibrar.)");
  } else {
    calibrarSensores();
  }
#endif
  pinMode(PINO_BOTAO, INPUT_PULLUP);

  Serial.println("Pronto! Enviando...");
  proximoEnvio = millis();
}

void loop() {
#if !USAR_A3144
  if (pediuCalibracao()) {
    calibrarSensores();
    proximoEnvio = millis();
  }
#endif
  if ((long)(millis() - proximoEnvio) < 0) return;
  proximoEnvio += PERIODO_MS;

  for (uint8_t i = 0; i < 5; i++) {
    leituraCrua[i] = lerSensor(i);
#if USAR_A3144
    float bruto = leituraCrua[i] ? 100.0 : 0.0;
    pctFiltrado[i] += ALFA * (bruto - pctFiltrado[i]);
    if (!dedoFechado[i] && pctFiltrado[i] > LIMIAR_FECHAR) dedoFechado[i] = true;
    if (dedoFechado[i] && pctFiltrado[i] < LIMIAR_ABRIR)   dedoFechado[i] = false;
    pacote.pct[i] = dedoFechado[i] ? 100 : 0;
#else
    fioSolto[i] = !leituraValida(leituraCrua[i]);
    int bruto = fioSolto[i] ? 0 : paraPorcento(i, leituraCrua[i]);   // fio solto: dedo abre
    pctFiltrado[i] += ALFA * (bruto - pctFiltrado[i]);
    pacote.pct[i] = (uint8_t)(pctFiltrado[i] + 0.5);
#endif
  }

  pacote.seq++;
  esp_err_t r = esp_now_send(ENDERECO_TODOS, (uint8_t*)&pacote, sizeof(pacote));
  if (r != ESP_OK) errosEnvio++;

  // LED: pisca curtinho a cada 16 pacotes (~3 vezes por segundo) = transmitindo
  digitalWrite(PINO_LED, (pacote.seq & 0x0F) == 0);

  // Depuracao 5 vezes por segundo
  if ((long)(millis() - proximoPrint) >= 0) {
    proximoPrint = millis() + 200;
    Serial.printf("cru: %4d %4d %4d %4d %4d | %%: %3d %3d %3d %3d %3d | seq %3u | erros %lu\n",
                  leituraCrua[0], leituraCrua[1], leituraCrua[2], leituraCrua[3], leituraCrua[4],
                  pacote.pct[0], pacote.pct[1], pacote.pct[2], pacote.pct[3], pacote.pct[4],
                  pacote.seq, errosEnvio);
  }
#if !USAR_A3144
  if ((long)(millis() - proximoAvisoFio) >= 0) {
    proximoAvisoFio = millis() + 2000;
    for (uint8_t i = 0; i < 5; i++) {
      if (fioSolto[i]) Serial.printf("!! FIO SOLTO? sensor do %s\n", NOME_DEDO[i]);
    }
  }
#endif
}
```

receptor_esp32/receptor_esp32.ino

ESP32 DevKit V1 junto da mão · versão 2 · 167 linhas

Recebe o ESP-NOW e repassa um quadro de 7 bytes ao Uno pelo TX2 (GPIO17).

```
/*
  ============================================================================
  MAO ROBOTICA - VERSAO 2 (SEM FIO) - RECEPTOR  -  ESP32 DevKit V1
  ============================================================================
  O que faz:
    Fica junto da mao robotica. Recebe pelo ar (ESP-NOW) os pacotes da
    luva e repassa para o Arduino Uno por um fio (serial), 50 vezes por
    segundo. Quem mexe os servos continua sendo o Uno.

  Ligacoes:
    ESP32 TX2 (GPIO17)  ->  pino D0 (RX) do Uno (fio direto, SEM resistor
                            e SEM divisor de tensao)
    ESP32 GND           ->  GND do Uno          (OBRIGATORIO!)
    Alimente este ESP32 pela USB (ou 5 V no pino VIN, mas nunca USB e VIN
    ao mesmo tempo). NAO alimente pelo pack de pilhas dos servos: quando os
    servos puxam corrente a tensao cai e o ESP32 reinicia.
    So existe um fio de dados: o ESP32 fala e o Uno so escuta.
    (3,3 V do ESP32 e suficiente para o Uno entender como "1".)
    NUNCA ligue o D1 (TX) do Uno aqui: 5 V queimam o ESP32.

  Este codigo e para o ESP32 "classico" (DevKit V1, modulo ESP-WROOM-32).
  ESP32-S3/C3/C6 tem outros pinos. Em placas WROVER (com PSRAM) o GPIO16 e
  o GPIO17 sao usados pela memoria: troque PINO_TX_UNO para 25, por exemplo.

  Quadro enviado ao Uno (7 bytes):
    0xFF, p0, p1, p2, p3, p4, xor(p0..p4)
    p0..p4 = fechamento 0..100 % (polegar, indicador, medio, anelar, minimo)
    Como p sempre e menor que 128, o 0xFF so aparece no comeco do quadro.

  LED azul (GPIO2): aceso = recebendo da luva | apagado = sem sinal.
  Se a luva parar de mandar por 300 ms, este ESP32 para de mandar para o
  Uno, e o Uno abre a mao sozinho (seguranca).

  Placa na IDE: "ESP32 Dev Module" (ou "DOIT ESP32 DEVKIT V1").
  ============================================================================
*/

#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>
#include <esp_arduino_version.h>

const uint8_t CANAL_WIFI = 1;          // MESMO canal da luva
const uint8_t PINO_LED = 2;
const int     PINO_TX_UNO = 17;        // TX2 -> D0 do Uno
const int     PINO_RX_NAO_USADO = 16;  // RX2 (nao precisa ligar nada)
const unsigned long PERIODO_MS = 20;   // 50 quadros por segundo
const unsigned long TEMPO_SEM_SINAL_MS = 300;

// ---------------- Pacote recebido (IGUAL na luva!) ----------------
// Se houver OUTRA mao robotica com este mesmo programa por perto (feira de
// ciencias), uma luva vai mexer a mao da outra! Para evitar, troque a letra
// de MAGIC_2 (ex.: 'X') AQUI e na luva_esp32 (e, se quiser, o canal).
const uint8_t MAGIC_1 = 'M';
const uint8_t MAGIC_2 = 'R';
typedef struct __attribute__((packed)) {
  uint8_t magic[2];   // 'M','R'
  uint8_t seq;
  uint8_t pct[5];
} PacoteMao;

// Dados compartilhados entre a funcao de recepcao (roda na tarefa do
// Wi-Fi) e o loop(). A "trava" (spinlock) impede ler no meio de uma copia.
portMUX_TYPE trava = portMUX_INITIALIZER_UNLOCKED;
PacoteMao ultimoPacote;
```

```
unsigned long momentoUltimoPacote = 0;
bool jaRecebeu = false;
unsigned long pacotesRecebidos = 0;

unsigned long proximoEnvio = 0;
unsigned long proximoPrint = 0;

// Chamada automaticamente quando chega um pacote ESP-NOW.
// Tem que ser RAPIDA: so confere e copia. Nada de delay ou Serial aqui.
// O formato mudou no core 3.x do ESP32, por isso o #if.
#if ESP_ARDUINO_VERSION_MAJOR >= 3
void aoReceber(const esp_now_recv_info_t* info, const uint8_t* dados, int tamanho) {
#else
void aoReceber(const uint8_t* mac, const uint8_t* dados, int tamanho) {
#endif
  if (tamanho != sizeof(PacoteMao)) return;          // tamanho errado: ignora
  if (dados[0] != MAGIC_1 || dados[1] != MAGIC_2) return; // nao e da nossa luva
  portENTER_CRITICAL(&trava);
  memcpy(&ultimoPacote, dados, sizeof(PacoteMao));
  momentoUltimoPacote = millis();
  jaRecebeu = true;
  pacotesRecebidos++;
  portEXIT_CRITICAL(&trava);
}

void setup() {
  Serial.begin(115200);                                          // USB (depuracao)
  // Para o Uno. Deixe os pinos escritos aqui: no core 3.x o Serial2 sem
  // pinos usa outros GPIO. 115200 tem que ser IGUAL ao Serial.begin do Uno.
  Serial2.begin(115200, SERIAL_8N1, PINO_RX_NAO_USADO, PINO_TX_UNO);
  pinMode(PINO_LED, OUTPUT);
  delay(300);
  Serial.println("\n=== Mao Robotica - Receptor (ESP32) ===");

  WiFi.mode(WIFI_STA);
  WiFi.disconnect();
  if (esp_wifi_set_channel(CANAL_WIFI, WIFI_SECOND_CHAN_NONE) != ESP_OK) {
    Serial.println("ERRO: nao consegui mudar o canal do Wi-Fi!");
  }
  uint8_t canal = 0;
  wifi_second_chan_t canal2;
  esp_wifi_get_channel(&canal, &canal2);
  Serial.printf("Canal Wi-Fi: %u (tem que ser igual ao da luva)\n", canal);

  uint8_t mac[6];
  esp_wifi_get_mac(WIFI_IF_STA, mac);
  Serial.printf("MAC deste receptor: %02X:%02X:%02X:%02X:%02X:%02X\n",
                mac[0], mac[1], mac[2], mac[3], mac[4], mac[5]);

  if (esp_now_init() != ESP_OK) {
    Serial.println("ERRO: nao consegui iniciar o ESP-NOW. Reiniciando...");
    delay(2000);
    ESP.restart();
  }
  esp_now_register_recv_cb(aoReceber);

  Serial.println("Esperando a luva...");
  proximoEnvio = millis();
}

void loop() {
  if ((long)(millis() - proximoEnvio) < 0) return;
  proximoEnvio += PERIODO_MS;

  // Copia os dados com a trava fechada (rapidinho)
  PacoteMao p;
  unsigned long momento;
  bool recebeu;
```

```
  portENTER_CRITICAL(&trava);
  p = ultimoPacote;
  momento = momentoUltimoPacote;
  recebeu = jaRecebeu;
  unsigned long total = pacotesRecebidos;
  portEXIT_CRITICAL(&trava);

  bool linkOk = recebeu && (millis() - momento < TEMPO_SEM_SINAL_MS);
  digitalWrite(PINO_LED, linkOk ? HIGH : LOW);

  if (linkOk) {
    uint8_t quadro[7];
    uint8_t x = 0;
    quadro[0] = 0xFF;                       // cabecalho
    for (uint8_t i = 0; i < 5; i++) {
      uint8_t v = p.pct[i] > 100 ? 100 : p.pct[i];
      quadro[1 + i] = v;
      x ^= v;                               // "ou exclusivo" para conferir
    }
    quadro[6] = x;
    Serial2.write(quadro, sizeof(quadro));
  }
  // Sem sinal: nao manda nada. O Uno percebe e abre a mao.

  if ((long)(millis() - proximoPrint) >= 0) {
    proximoPrint = millis() + 500;
    if (linkOk) {
      Serial.printf("OK  seq %3u | %%: %3u %3u %3u %3u %3u | pacotes %lu\n",
                    p.seq, p.pct[0], p.pct[1], p.pct[2], p.pct[3], p.pct[4], total);
    } else {
      Serial.println("SEM SINAL da luva (nao estou mandando nada ao Uno)");
    }
  }
}
```

mao_sem_fio/mao_sem_fio.ino

Arduino Uno · versão 2 · 256 linhas

Lê os quadros no D0, confere o XOR e move os servos. Abre a mão se os dados pararem por 500 ms. Botão no D2 (segurar 2 s) liga o modo demonstração.

```
/*
  ============================================================================
  MAO ROBOTICA - VERSAO 2 (SEM FIO) - MAO  -  Arduino Uno
  ============================================================================
  !!! ATENCAO: DESCONECTE O FIO DO PINO D0 ANTES DE GRAVAR ESTE PROGRAMA !!!
  (O D0 e o mesmo pino que a USB usa para gravar. Com o ESP32 ligado nele,
   a gravacao falha. Grave, depois ligue o fio de novo.)

  O que faz:
    Recebe do ESP32 receptor (pelo pino D0) quanto cada dedo da luva esta
    fechado (0 a 100 %) e move os servos da mao robotica.
    Se ficar 500 ms sem receber nada certo, ABRE a mao (seguranca).

  Ligacoes:
    ESP32 receptor GPIO17 (TX2) --> D0 (RX) do Uno  (fio direto, SEM resistor
                                    e SEM divisor de tensao nesse fio)
    ESP32 GND                   --> GND do Uno (OBRIGATORIO)
    NUNCA ligue o D1 (TX) do Uno no ESP32: os 5 V do Uno queimam o ESP32.
    Nao deixe o fio no D0 com o ESP32 desligado (ligue os dois juntos).
    Servos SG90 (sinal):  polegar = D3, indicador = D5, medio = D6,
                          anelar = D9, minimo = D10
    Pack 4 pilhas AA (4,8 V): + --> vermelho dos servos,
                              - --> marrom dos servos E GND do Uno.
```

```
  Quadro esperado (7 bytes, 115200 baud):
    0xFF, p0, p1, p2, p3, p4, xor(p0..p4)    (p = 0..100)

  LED do pino 13: aceso = chegando dados bons | apagado = sem sinal |
                  piscando = modo demonstracao.

  MODO DEMONSTRACAO (plano B se o radio falhar na apresentacao):
    botao entre D2 e GND (opcional). Segure 2 s para ligar ou desligar.
    A mao faz gestos sozinha e ignora a luva.
  O Monitor Serial (115200) mostra o que chegou. NAO digite nada nele
  enquanto o ESP32 estiver ligado no D0.
  ============================================================================
*/

#include <Servo.h>

// ---------------- Pinos (ordem: polegar, indicador, medio, anelar, minimo)
const uint8_t PINO_SERVO[5] = { 3, 5, 6, 9, 10 };
const uint8_t PINO_LED = 13;
const uint8_t PINO_BOTAO = 2;   // opcional: botao entre D2 e GND

// ---------------- Calibracao dos servos (mesmos numeros da versao 1) -----
// Copie do "teste_servos"/"mao_com_fio". Sempre entre 0 e 180.
int ANG_ABERTO[5]  = { 20, 20, 20, 20, 20 };     // dedo esticado
int ANG_FECHADO[5] = { 160, 160, 160, 160, 160 }; // dedo dobrado

// ---------------- Ajustes finos ----------------
const unsigned long PERIODO_MS = 20;        // 50 vezes por segundo
const int   ZONA_MORTA = 2;                 // graus
const int   PASSO_MAX  = 6;                 // graus por ciclo (limite de velocidade)
const unsigned long TEMPO_FAILSAFE_MS = 500; // sem dados por 500 ms -> abre a mao
const unsigned long APERTO_LONGO_MS = 2000;  // segurar o botao 2 s = modo demonstracao

// ---------------- Gestos do modo demonstracao (iguais aos da versao 1) ----
// Fechamento de cada dedo em % (polegar, indicador, medio, anelar, minimo).
struct Gesto {
  uint8_t pct[5];
  uint16_t ms;
};
const Gesto GESTOS[] = {
  { {   0,   0,   0,   0,   0 }, 1500 },  // mao aberta
  { { 100, 100, 100, 100, 100 }, 1500 },  // punho
  { { 100,   0, 100, 100, 100 }, 1000 },  // conta 1
  { { 100,   0,   0, 100, 100 }, 1000 },  // conta 2
  { { 100,   0,   0,   0, 100 }, 1000 },  // conta 3
  { { 100,   0,   0,   0,   0 }, 1000 },  // conta 4
  { {   0,   0,   0,   0,   0 }, 1200 },  // conta 5
  { {   0, 100, 100, 100, 100 }, 1800 },  // joinha
  { { 100,   0, 100, 100,   0 }, 1800 },  // rock
  { {  70,  70,   0,   0,   0 }, 1800 },  // pinca / "ok"
  { {   0,   0,   0,   0, 100 },  500 },  // onda: fecha do minimo ao polegar...
  { {   0,   0,   0, 100, 100 },  500 },
  { {   0,   0, 100, 100, 100 },  500 },
  { {   0, 100, 100, 100, 100 },  500 },
  { { 100, 100, 100, 100, 100 },  800 },
  { { 100, 100, 100, 100,   0 },  500 },  // ...e abre na mesma ordem
  { { 100, 100, 100,   0,   0 },  500 },
  { { 100, 100,   0,   0,   0 },  500 },
  { { 100,   0,   0,   0,   0 },  500 },
};
const uint8_t NUM_GESTOS = sizeof(GESTOS) / sizeof(GESTOS[0]);

// ---------------- Variaveis ----------------
Servo servo[5];
int angAtual[5];
int alvo[5];
int pctRecebido[5];             // ultimo quadro valido
```

```
unsigned long momentoUltimoQuadro = 0;
bool jaRecebeu = false;
unsigned long quadrosBons = 0;
unsigned long quadrosRuins = 0;

// Maquina de estados do leitor de quadros
uint8_t estado = 0;             // 0 = esperando 0xFF, 1 = lendo dedos, 2 = lendo xor
uint8_t bufferDedos[5];
uint8_t posicao = 0;

bool modoDemo = false;
uint8_t gestoAtual = 0;
unsigned long inicioGesto = 0;
bool botaoAntes = false;
unsigned long inicioAperto = 0;
bool apertoLongoFeito = false;

unsigned long proximoCiclo = 0;
unsigned long proximoPrint = 0;

// Le os bytes que chegaram e monta o quadro. Retorna true se fechou um quadro bom.
bool lerQuadros() {
  bool chegouQuadro = false;
  while (Serial.available() > 0) {
    uint8_t b = Serial.read();

    if (b == 0xFF) {            // 0xFF sempre e comeco de quadro
      if (estado != 0) quadrosRuins++;   // quadro anterior ficou pela metade
      estado = 1;
      posicao = 0;
      continue;
    }

    if (estado == 1) {          // lendo os 5 dedos
      if (b > 100) {            // valor impossivel: descarta
        quadrosRuins++;
        estado = 0;
        continue;
      }
      bufferDedos[posicao++] = b;
      if (posicao == 5) estado = 2;
    } else if (estado == 2) {   // byte de conferencia (xor)
      uint8_t x = 0;
      for (uint8_t i = 0; i < 5; i++) x ^= bufferDedos[i];
      if (x == b) {
        for (uint8_t i = 0; i < 5; i++) pctRecebido[i] = bufferDedos[i];
        momentoUltimoQuadro = millis();
        jaRecebeu = true;
        quadrosBons++;
        chegouQuadro = true;
      } else {
        quadrosRuins++;
      }
      estado = 0;
    }
    // estado 0 e byte diferente de 0xFF: lixo, ignora
  }
  return chegouQuadro;
}

// Leva o servo i na direcao do fechamento pct (0..100 %),
// com zona morta e limite de velocidade. (Igual a versao 1.)
void atualizarServo(uint8_t i, int pct) {
  int alvoNovo = map(pct, 0, 100, ANG_ABERTO[i], ANG_FECHADO[i]);
  bool extremo = (pct == 0 || pct == 100);
  if (abs(alvoNovo - alvo[i]) > ZONA_MORTA || extremo) {
    alvo[i] = alvoNovo;
  }
```

```
  int falta = alvo[i] - angAtual[i];
  if (falta ≠ 0) {
    angAtual[i] += constrain(falta, -PASSO_MAX, PASSO_MAX);
    servo[i].write(angAtual[i]);
  }
}

// Botao entre D2 e GND (INPUT_PULLUP: apertado = LOW). Segurar 2 s liga
// ou desliga a demonstracao. Sem botao ligado, nada acontece.
void lerBotao() {
  bool apertado = (digitalRead(PINO_BOTAO) == LOW);
  unsigned long agora = millis();
  if (apertado && !botaoAntes) {
    inicioAperto = agora;
    apertoLongoFeito = false;
  }
  if (apertado && !apertoLongoFeito && agora - inicioAperto ≥ APERTO_LONGO_MS) {
    apertoLongoFeito = true;
    modoDemo = !modoDemo;
    gestoAtual = 0;
    inicioGesto = agora;
    Serial.println(modoDemo ? F(">> MODO DEMONSTRACAO ligado") : F(">> MODO DEMONSTRACAO desligado"));
  }
  botaoAntes = apertado;
}

void setup() {
  // 115200 = o MESMO valor do Serial2.begin do receptor_esp32.
  // (O Uno a 16 MHz gera 115200 com 2,1 % de erro; costuma funcionar. Se o
  //  contador "ruins" crescer sem parar, troque para 38400 AQUI, no
  //  receptor_esp32 E no Monitor Serial.)
  Serial.begin(115200);
  pinMode(PINO_LED, OUTPUT);
  pinMode(PINO_BOTAO, INPUT_PULLUP);
  Serial.println(F("\n=== Mao Robotica - Versao 2 (sem fio) - Uno ==="));

  // Liga os servos UM DE CADA VEZ, ja com a mao aberta.
  for (uint8_t i = 0; i < 5; i++) {
    servo[i].write(ANG_ABERTO[i]);
    servo[i].attach(PINO_SERVO[i]);
    angAtual[i] = ANG_ABERTO[i];
    alvo[i] = ANG_ABERTO[i];
    pctRecebido[i] = 0;
    delay(300);
  }

  // Joga fora o que chegou enquanto ligava os servos
  while (Serial.available() > 0) Serial.read();

  Serial.println(F("Esperando dados do ESP32 no pino D0..."));
  proximoCiclo = millis();
}

void loop() {
  lerQuadros();    // le sempre, para o buffer da serial nao encher
  lerBotao();

  if ((long)(millis() - proximoCiclo) < 0) return;
  proximoCiclo += PERIODO_MS;

  bool sinalOk = jaRecebeu && (millis() - momentoUltimoQuadro < TEMPO_FAILSAFE_MS);

  if (modoDemo) {
    digitalWrite(PINO_LED, (millis() / 250) % 2);   // pisca = demonstracao
    if (millis() - inicioGesto ≥ GESTOS[gestoAtual].ms) {
      gestoAtual = (gestoAtual + 1) % NUM_GESTOS;
      inicioGesto = millis();
    }
```

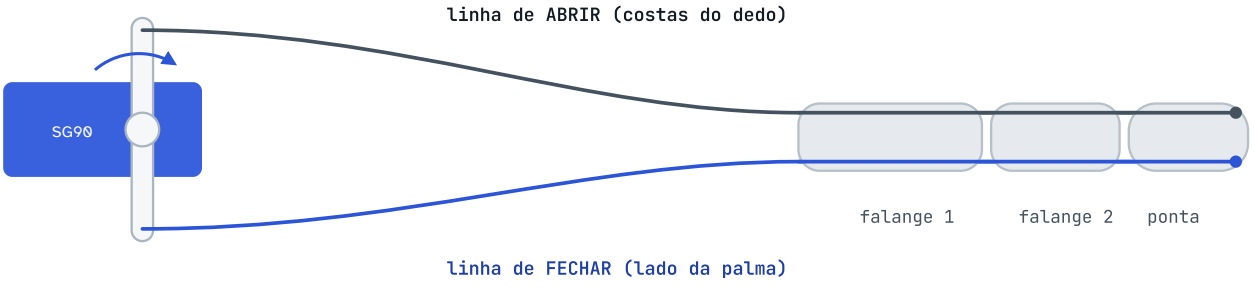
```
    for (uint8_t i = 0; i < 5; i++) atualizarServo(i, GESTOS[gestoAtual].pct[i]);
  } else {
    digitalWrite(PINO_LED, sinalOk ? HIGH : LOW);
    for (uint8_t i = 0; i < 5; i++) {
      int pct = sinalOk ? pctRecebido[i] : 0;    // sem sinal = mao aberta
      atualizarServo(i, pct);
    }
  }

  // Mostra no Monitor Serial 4 vezes por segundo
  if ((long)(millis() - proximoPrint) >= 0) {
    proximoPrint = millis() + 250;
    if (modoDemo) {
      Serial.print(F("DEMO gesto "));
      Serial.print(gestoAtual);
    } else if (sinalOk) {
      Serial.print(F("OK  %:"));
      for (uint8_t i = 0; i < 5; i++) { Serial.print(' '); Serial.print(pctRecebido[i]); }
      Serial.print(F(" | ang:"));
      for (uint8_t i = 0; i < 5; i++) { Serial.print(' '); Serial.print(angAtual[i]); }
    } else {
      Serial.print(F("SEM SINAL -> mao aberta"));
    }
    Serial.print(F(" | bons "));
    Serial.print(quadrosBons);
    Serial.print(F(" ruins "));
    Serial.println(quadrosRuins);
  }
}
```

MECÂNICA

Tendões e dedos

Pela foto, o kit segue o modelo de mão com tendões: os servos ficam no antebraço e puxam linhas que passam por dentro dos dedos.



O braço do servo gira: puxa uma linha e solta a outra.
Se o kit usa elástico para abrir, só existe a linha de fechar.

Duas linhas por dedo. Com um braço duplo no servo, a mesma rotação fecha e abre o dedo. Monte o braço com o servo já em 90°, que é o meio do curso. Assim sobra curso para os dois lados.

Montagem

1. Antes de prender o braço, rode o `teste_servos` para deixar todos em 90°.
2. Use linha de pesca **multifilamento** (trançada). A de nylon comum estica com o tempo.
3. Na ponta do dedo, dê 3 ou 4 nós e pingue uma gota de super cola. Esse é o método do InMoov, o projeto de mão robótica impressa mais conhecido. No servo, puxe a linha bem esticada antes de apertar o parafuso que a prende.
4. Não deixe as linhas se torcerem ou se cruzarem no caminho até o dedo. Linha torcida trava o dedo.
5. Ajuste a tensão com o dedo totalmente aberto e a linha de fechar sem folga.
6. Depois descubra os ângulos reais de “aberto” e “fechado” de cada dedo e anote no código.

O que costuma dar errado

- **Dedo não fecha todo:** falta curso. Mude o furo do braço (mais longe do centro puxa mais linha) ou o ângulo.
- **Servo zumbindo parado:** ele está forçando contra o dedo. Diminua `ANG_FECHADO` alguns graus.
- **Dedo duro:** a linha está raspando na impressão 3D. Lixe os furos ou passe a linha por um tubinho de PTFE ou de cotonete.
- **Polegar:** no kit ele provavelmente só dobra num eixo. Aceite isso e não tente pinça de precisão.

ORDEM

Ordem de montagem, sem pular etapa

Cada etapa testa uma coisa só. Se você montar tudo de uma vez e não funcionar, não vai saber se o problema é energia, sensor, código ou mecânica.

Calendário até 10 de novembro

QUANDO	O QUE FAZER	ETAPAS ABAIXO
qua 21 – sex 23/out	**Comprar tudo.** Prefira anúncios com entrega rápida (“chega amanhã”, Full). Confirme que o kit sai do Brasil. Instale a Arduino IDE e leia este guia inteiro.	—
sáb 24 – ter 27/out	Enquanto as peças não chegam: teste o que você já tem (Uno + 1 servo), prepare a luva e escreva o roteiro da apresentação.	—
qua 28 – qui 29/out	Um servo e um sensor. Depois os 5 servos nas pilhas.	1–2
sex 30/out – seg 2/nov	Montar a mão, passar os tendões e calibrar os ângulos. O dia 2 é feriado (Finados), então aproveite.	3–4

QUANDO	O QUE FAZER	ETAPAS ABAIXO
ter 3 – qua 4/nov	Luva pronta e versão 1 completa.	5–6
qua 4/nov	● DATA DE CORTE A versão 1 funciona 10 minutos sem travar? Se não, esqueça o ESP32 e use o resto do tempo para deixar a versão 1 perfeita.	—
qui 5 – sáb 7/nov	Versão 2, só se você passou na data de corte.	7–8
dom 8 – seg 9/nov	Ensaio completo, folga para imprevistos e pilhas carregadas.	—
ter 10/nov	Apresentação. Leve o cabo da versão 1, pilhas reserva e um servo reserva.	—

COMPRAR OU NÃO OS ESP32 AGORA?

Se você esperar a versão 1 ficar pronta para só então comprar os ESP32, eles não chegam a tempo. Então compre **agora** se o dinheiro (uns R$ 80–160) não fizer falta, sabendo que talvez nem use. Se o dinheiro for apertado, não compre e faça só a versão 1 bem feita. Uma mão com fio que funciona vale mais que uma sem fio que trava.

1 ## Um servo, um sensor, na protoboard

Uno na USB, 1 servo nas pilhas com GND comum, 1 SS49E em A0. Rode `mao_com_fio` só com o polegar ligado.

Pronto quando: o servo vira quando você aproxima o ímã do sensor.

2 ## Cinco servos com as pilhas

Ligue os 5 servos e rode `teste_servos`. Mande todos para 0° e 180° (sem a mão montada).

Pronto quando: os 5 se mexem juntos e o Uno não reinicia.

3 ## Mecânica de um dedo

Monte o indicador com tendões e calibre `ANG_ABERTO` e `ANG_FECHADO` dele.

Pronto quando: o dedo abre e fecha inteiro sem o servo zumbir.

4 ## Os outros quatro dedos

Repita a calibração para cada dedo e anote os ângulos.

Pronto quando: todos abrem e fecham com o `teste_servos`.

5 ## A luva

Faça o teste da fita com o `teste_sensores`, dedo por dedo. Depois cole sensores e ímãs e faça o cabo de 7 fios.

Pronto quando: cada dedo vai de ~0 % a ~100 % acompanhando o movimento inteiro, sem mexer muito os vizinhos.

6 Versão 1 completa

Luva + Uno + mão. Ensaie a demonstração.

Pronto quando: funciona 10 minutos seguidos sem travar. **Aqui o projeto já está apresentável.**

7 Rádio na bancada (versão 2)

Dois ESP32 na mesa, sem sensores: grave os sketches e veja no Monitor Serial do receptor os pacotes chegando.

Pronto quando: o LED do receptor fica aceso e apaga quando você desliga a luva.

8 Versão 2 completa

Sensores no ESP32 da luva, receptor no D0 do Uno, grave `mao_sem_fio`.

Pronto quando: funciona andando pela sala, e a mão abre sozinha quando a luva é desligada.

PROBLEMAS

Quando algo não funcionar

SINTOMA	CAUSA MAIS PROVÁVEL	O QUE FAZER
Uno reinicia quando os servos mexem	Servos puxando energia do Uno	Servos só nas pilhas. Confira se o vermelho de nenhum servo está no 5V do Uno.
Servos tremem ou se mexem sozinhos	Falta GND comum, ou pilhas fracas	Ligue GND do Uno na linha GND. Recarregue as pilhas. Coloque o capacitor.
Servo não se mexe	Fio laranja no pino errado, ou pilhas desligadas	Confira a tabela de pinos. Rode o `teste_servos`.
Servo zumbe parado e esquenta	Ângulo além do que o dedo alcança	Reduza `ANG_FECHADO` (ou aumente `ANG_ABERTO`) alguns graus.
Sensor sempre no mesmo valor	Sensor ao contrário, sem VCC, ou ímã longe	Confira a pinagem pela face com a escrita. Meça a saída: sem ímã ela fica perto da metade de VCC.
Aparece “FIO SOLTO? sensor do …”	Leitura impossível (perto de 0 V ou 5 V): fio quebrado, solda fria ou sensor ao contrário	Esse dedo fica aberto até você consertar. Confira VCC, GND e OUT daquele sensor com o `teste_sensores`.
Dedo calibrado ontem está errado hoje	A luva foi vestida um pouco diferente	Calibre de novo toda vez que vestir a luva: um aperto curto no botão (ou `c` no Monitor).
Leitura muda sem você mexer o dedo	Luva escorregando sobre o nó	Use luva mais justa ou prenda a luva com velcro no punho e entre os dedos.
Dedo errado fecha	Ímã vizinho perto demais, ou fios trocados	Teste um dedo por vez no Monitor Serial. Use ímã menor ou afaste o sensor.

SINTOMA	CAUSA MAIS PROVÁVEL	O QUE FAZER
A3144 não reage	Polo errado do ímã, ou sem pull-up	Vire o ímã. No ESP32 coloque o resistor de 10 kΩ.
ESP32: leitura sempre 0 ou 4095	Pino do ADC2 com rádio ligado	Use só os pinos 32 a 39.
Uno não grava (“avrdude: not in sync”)	Fio no D0 ligado	Solte o fio do D0, grave e ligue de novo.
Versão 2: o contador “ruins” do Uno não para de subir	Erro de velocidade da serial (115200 no Uno tem 2 % de erro)	Troque para 38400 no `mao_sem_fio`, no `receptor_esp32` e no Monitor Serial.
Mão abre sozinha na versão 2	Failsafe: o Uno parou de receber dados	Veja o LED do receptor. Recarregue o power bank. Aproxime a luva. Não alimente o receptor pelas pilhas dos servos: quando eles puxam corrente, o ESP32 reinicia.
Power bank desliga sozinho	Consumo baixo demais para ele	Use um power bank sem desligamento automático (alguns têm modo para baixa corrente) ou um módulo 18650 com regulador.
ESP32 fica em “Connecting...” e não grava	A placa não entrou no modo de gravação	Segure o botão BOOT enquanto aparece “Connecting...” e solte quando começar a gravar.
ESP32 não aparece na porta USB	Cabo só de carga, ou falta driver	Teste outro cabo. Instale o driver CP210x ou CH340, conforme o chip da placa.

DIA DA APRESENTAÇÃO

Roteiro, kit de emergência e checklist

Na feira tudo dá errado de um jeito novo: luz forte, mesa bamba, gente esbarrando, Wi-Fi lotado. Ensaie este roteiro pelo menos 3 vezes em casa.

SLIDES PRONTOS

Existe uma apresentação de 12 slides feita para este projeto, com notas do que falar em cada um: Mão Robótica: Apresentação. Troque os campos entre colchetes ([Seu nome], [Escola], as fotos e os números do slide de resultados) pelos seus. Ela exporta para PowerPoint e PDF.

TEMPO	O QUE MOSTRAR	O QUE FALAR
0:00–0:30	A mão parada, aberta	O problema: controlar uma mão robótica com os movimentos da sua própria mão.
0:30–1:00	Botão de 2 s: modo demonstração	“A mão tem 5 servos, um por dedo, que puxam tendões como os nossos.” Enquanto ela conta de 1 a 5 e faz joinha.

TEMPO	O QUE MOSTRAR	O QUE FALAR
1:00–1:30	Vestir a luva e calibrar (aperto curto, 3 posições)	“Cada pessoa tem uma mão diferente, então o sistema aprende a sua em 10 segundos.”
1:30–2:30	Controlar a mão: dedo por dedo, devagar, depois punho	Mostre que o movimento é proporcional: o dedo para no meio se o seu parar no meio.
2:30–3:00	Notebook com o **Serial Plotter** (comando `p`)	“Isto é o que o Arduino vê: uma curva por dedo.” Fecha mostrando que você entende o sinal.

Kit de emergência na mochila

- 4 pilhas NiMH **carregadas na véspera** + 4 de reserva
- Cabo USB do Uno + power bank (caso não tenha tomada nem notebook)
- 1 servo SG90 de reserva, já com o braço (horn) e no ângulo de 90°
- Chave de fenda pequena, linha de pesca, super cola, fita isolante
- Jumpers sobrando e 2 sensores SS49E de reserva
- Notebook com a Arduino IDE e os sketches com os **seus** ângulos já salvos
- Versão 2: o cabo da versão 1 e o firmware `mao_com_fio` pronto para gravar

Checklist antes de começar

1. Chave das pilhas ligada, Uno ligado, LED 13 apagado (pronto).
2. Modo demonstração uma vez completo: todos os dedos abrem e fecham sem zumbir.
3. Vestir a luva e calibrar ali, na mesa.
4. Testar um dedo de cada vez.
5. Longe de caixas de som, ímãs de geladeira e celulares em cima da luva: tudo isso mexe no campo magnético.
6. Desligar as pilhas entre uma apresentação e outra, para os servos não ficarem forçando.

APRESENTAÇÃO

Perguntas que a banca provavelmente vai fazer

Treine estas respostas. Se você souber responder, a banca percebe que você entende o projeto e não só montou.

- **Por que usar ESP32 se o Arduino já faz tudo?**

  “Para tirar o fio entre o operador e a mão. O Uno continua controlando os servos. O ESP32 só cuida da comunicação sem fio, que o Uno não tem. Isso permite operar a mão à distância, como em

robôs de resgate ou de laboratório.”

- **Por que não ligou os servos no 5V do Arduino?**

  “Cada SG90 pode puxar centenas de miliamperes quando força, e cinco passam de 3 A. O 5V do Uno na USB fornece uns 500 mA. Por isso os servos têm pilhas próprias, com o GND ligado ao do Arduino para o sinal ter a mesma referência.”

- **Como funciona o sensor Hall?**

  “Quando passa corrente por uma plaquinha de semicondutor dentro de um campo magnético, aparece uma pequena tensão nos lados dela. Esse é o efeito Hall. O SS49E amplifica essa tensão e entrega um valor proporcional ao campo.”

- **Por que o sensor fica no nó do dedo, e não na palma?**

  “O campo do ímã cai mais ou menos com o cubo da distância. Se o ímã só se aproximasse do sensor, o sinal apareceria apenas no último centímetro e o dedo fecharia de uma vez. Com um de cada lado do nó, a poucos milímetros, o ímã gira e se afasta aos poucos quando o dedo dobra, e o sinal acompanha o movimento inteiro. Como o sensor fica nas costas da mão, o fio dele não passa por nenhuma articulação.” Se perguntarem a limitação: “ele mede a dobra da base do dedo, não das pontas.”

- **E se a luva parar de funcionar na hora?**

  Segure o botão por 2 segundos: a mão entra no modo demonstração e faz gestos sozinha (conta de 1 a 5, joinha, rock, onda). Você continua apresentando a mecânica e o controle dos servos enquanto confere a luva. Treine essa troca antes.

- **O que acontece se o sinal sem fio cair?**

  “Se o Uno fica meio segundo sem receber dados válidos, ele abre a mão sozinho. Isso é um failsafe: uma falha de comunicação nunca deixa a mão travada apertando alguma coisa.”

- **Como o Uno sabe que os dados chegaram certos?**

  “Cada pacote começa com um byte de cabeçalho e termina com uma soma de verificação (XOR). Se a conta não bate, o pacote é descartado.”

- **Quanto custou?**

  Tenha o número na ponta da língua, somando a tabela da seção Compras.

Fontes e créditos

Datasheets e documentação conferidos para este guia

- TowerPro SG90, datasheet · tensão, torque e pulso
- MG90S, especificações
- Esquemático do Arduino Uno Rev3 · fusível de 500 mA, regulador NCP1117 e resistor de 1 kΩ no D0
- Biblioteca Servo.h · 12 servos no Uno, PWM desligado em D9/D10
- ATmega328P, datasheet · VIH = 0,6 × VCC
- SoftwareSerial (código-fonte) · por que ele faz o servo tremer
- Honeywell SS49E, datasheet · 2,7–6,5 V, 1,4 mV/G, ±1000 G
- Allegro A3141–A3144, datasheet · 4,5–24 V, BOP 70–350 G
- ESP-IDF: ADC · ADC2 não funciona com Wi-Fi
- ESP-IDF: ESP-NOW · broadcast, canal e callbacks
- Arduino-ESP32: HardwareSerial · pinos padrão da Serial2 mudaram no core 3.x
- ESP32, datasheet · consumo com rádio
- Adafruit: capacitor na alimentação dos servos
- Power banks que desligam com consumo baixo
- Luva de baixo custo com sensores Hall por articulação (arXiv, 2026) · mesmo princípio da posição no nó do dedo
- Tese Chalmers: sensores Hall para medir os dedos
- Amplitude de movimento normal dos dedos (Indian J Plast Surg, 2024)
- InMoov: como passar e tensionar os tendões
- Projeto de luva com Hall e mão com servos (Univ. Illinois, ECE 445)
- Projeto de mão robótica do IFBA (2025) · referência de um projeto parecido

Os campos magnéticos do gráfico foram calculados com a fórmula do campo no eixo de um cilindro magnetizado (Br ≈ 1,19 T para N35). Não há medição publicada para esses ímãs, então meça os seus.

Imagens

- Foto do kit: captura do anúncio no Mercado Livre enviada por você.
- Arduino Uno R3, micro servo 9 g e protoboard: projeto Fritzing (fritzing-parts), licença CC BY-SA 3.0. Peças “Arduino Uno (Rev3)”, “Dagu DGServo 9g (Top view)” e “Half-Breadboard”, de seus autores no projeto Fritzing.
- Sensor Hall: derivado da peça “TMP36” do Fritzing (Jonathan Cohen), com a marcação trocada para “49E”. CC BY-SA 3.0.
- Suporte de pilhas: derivado de “4 x AAA Battery Mount” do Fritzing, reetiquetado como AA. Capacitor: derivado de “Electrolytic Capacitor” do Fritzing (Brendan Howell), pernas ajustadas. Ambos CC BY-SA 3.0.
- ESP32 DevKit V1: peça “LuaNode32 / ESP32” de G. Noorlander (TD-er/fritzing-parts), com o texto do módulo alterado. Licença MIT (texto abaixo).
- Ímã e todos os diagramas de ligação: desenhados para este guia.

▼ Texto da licença MIT da peça do ESP32

```
The MIT License (MIT)
```

Copyright (c) 2015

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Guia feito para uso próprio do estudante. Os preços são estimativas e os valores dos componentes vêm dos datasheets citados. Confira sempre o datasheet da peça que você comprou, porque clones podem ter especificações diferentes.